

HEIGHT: Heterogeneous Interaction Graph Transformer for Robot Navigation in Crowded and Constrained Environments

Shuijing Liu¹, Haochen Xia¹, Fatemeh Cheraghi Pouria¹, Kaiwen Hong¹,
Neeloy Chakraborty¹, *Graduate Student Member, IEEE*, Zichao Hu, *Member, IEEE*,
Joydeep Biswas, *Member, IEEE*, and Katherine Driggs-Campbell¹, *Member, IEEE*

Abstract—We study the problem of robot navigation in dense and interactive crowds with static constraints such as corridors and furniture. Previous methods fail to consider all types of spatial and temporal interactions among agents and obstacles, leading to unsafe and inefficient robot paths. In this article, we leverage a graph-based representation of crowded and constrained scenarios and propose a structured framework to learn robot navigation policies with deep reinforcement learning. We first split the representations of different inputs, and propose a heterogeneous spatio-temporal (st) graph to model distinct interactions among humans, robots, and obstacles. Based on the heterogeneous st-graph, we propose HEIGHT, a novel navigation policy network architecture with different components to capture heterogeneous interactions through space and time. HEIGHT utilizes attention mechanisms to prioritize important interactions and a recurrent network to track changes in the dynamic scene over time, encouraging the robot to avoid collisions adaptively. Through extensive simulation and real-world experiments, we demonstrate that HEIGHT outperforms state-of-the-art baselines in terms of success, navigation time, and generalization to domain shifts in challenging navigation scenarios.

Note to Practitioners—This work is motivated by the challenge of safe and efficient robot navigation in indoor environments with people, such as homes and offices. Conventional navigation methods make simplistic assumptions about interactions among agents and entities, which leads to unsafe or unnatural robot behavior in dynamic settings. Previous machine learning methods either ignore or use ineffective approaches to handle static obstacles—limiting their real-world applicability. For a practical system to reason about both dynamic people and static constraints simultaneously, the following two factors are critical: how the robot represents its surroundings and how it make decisions. To represent the dynamic environment, our method treats people and obstacles differently and keeps the

most essential information as inputs. On the architectural side, we propose a structured graph-based network, which consists of specialized models to explicitly models different types of interactions among all observed entities. In challenging simulation benchmarks, our approach outperforms alternative designs that are commonly used in existing works. We also demonstrate strong real-world deployment in everyday environments. To improve deployment, a future direction is continual learning in the real-world. More information is available at <https://sites.google.com/view/crowdnav-height/home>.

Index Terms—Navigation, mobile robots, artificial intelligence, neural networks, cooperative systems.

I. INTRODUCTION

ROBOTS are increasingly prevalent in human-centric environments. In addition to dynamic agents, real-world navigation environments usually contain static constraints such as walls, furniture, and untraversable terrains. In this article, we study robot navigation to a destination without colliding with humans and obstacles, a crucial ability for applications such as last-mile delivery and household robots. For example, Fig. 1 shows such a navigation scenario with abundant subtle interactions among the robot, humans, and obstacles. These interactions are heterogeneous, dynamic, and difficult to reason, making navigation in such environments challenging.

While a large body of research on robot crowd navigation studies open-world navigation and ignores static obstacles [1], [2], [3], some previous works have explored various approaches for crowd navigation in constrained spaces [4], [5], [6]. However, these works typically have one of two limitations: (1) They do not differentiate between dynamic and static obstacles, and thus the robot has difficulties taking adaptive strategies to avoid collisions [4], [6], [7], [8]; (2) The representations of scene, especially static obstacles, are inefficient or redundant as inputs to the navigation algorithm. To address these limitations, we ask the following research question: *How can a robot navigation policy represent and reason about diverse interactions in crowded and constrained environments to adaptively avoid collisions?*

To answer this question, we propose a framework that leverages the heterogeneity of interactions in crowded and constrained scenarios in both scene representation and navigation policy design. First, we use light weight perception modules to extract separate human and obstacle representations that

Received 1 May 2025; revised 8 August 2025 and 12 October 2025; accepted 13 November 2025. Date of publication 19 December 2025; date of current version 12 January 2026. This article was recommended for publication by Associate Editor J. Wang and Editor C. Yang upon evaluation of the reviewers' comments. This work was supported in part by the National Science Foundation (NSF) under Grant CAREER-2143435, Grant NSF CAREER-2046955, and Grant NSF SLES IIS-2416461; in part by Amazon; and in part by JP Morgan. (Haochen Xia and Fatemeh Cheraghi Pouria contributed equally to this work.) (Corresponding author: Shuijing Liu.)

Shuijing Liu, Zichao Hu, and Joydeep Biswas are with The University of Texas at Austin, Austin, TX 78722 USA (e-mail: shuijing.liu@utexas.edu).

Haochen Xia, Fatemeh Cheraghi Pouria, Kaiwen Hong, Neeloy Chakraborty, and Katherine Driggs-Campbell are with the University of Illinois Urbana-Champaign, Urbana, IL 61801 USA.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TASE.2025.3646588>, provided by the authors.

Digital Object Identifier 10.1109/TASE.2025.3646588



Fig. 1. A heterogeneous graph aids the robot's spatio-temporal reasoning in a crowded and constrained environment. The colored arrows denote robot-human (RH), human-human (HH), and obstacle-agent (OA) interactions. The opaque arrows are the more important interactions, while the transparent arrows are the less important ones. The white arrows indicate the front direction of the robot and its length indicates the robot speed. At each timestep t , the robot reasons about these interactions, focuses on the important ones, and makes decisions.

are essential for navigation, such as relative positions w.r.t. the robot. The split representations are processed and fed separately into the reinforcement learning (RL)-based navigation pipeline. Then, inspired by recent breakthroughs in spatio-temporal (st) networks for crowd navigation [2], [3], [8], [9], we decompose the scenario into a heterogeneous st-graph with different types of edges to represent different types of interactions among the robot, detected humans, and observed obstacles, as shown in the colored arrows in Fig. 1. Finally, we convert the heterogeneous st-graph into a **HE**terogeneous **I**nteraction **G**raph **T**ransformer (**HEIGHT**), a robot policy network consisting of different modules to parameterize the various spatio-temporal interactions. Specifically, we use two separate attention networks to address the different effects of robot-human (RH) and human-human (HH) interactions. The attention networks enable the robot to pay more attention to the important interactions, leading to a low collision-rate even as the number of humans increases.

In the rapidly changing scenario in Fig. 1, HEIGHT injects structures to and captures the synergy between scene representation and network architectures. By reasoning about the heterogeneous interactions among different components through space and time, the robot is able to avoid collisions and approach its goal in an efficient manner. In summary, the main contributions of this article are as follows.

- 1) We propose an input representation of crowded and constrained environments that treats observed humans and obstacles differently. The split scene representation naturally allows us to inject structures in the framework.
- 2) We propose a heterogeneous spatio-temporal (st) graph to effectively model the pairwise interactions among all agents and entities in crowded and constrained scenarios.
- 3) From the heterogeneous st-graph, we use a principle approach to derive HEIGHT, a transformer-based robot navigation policy network with different modules to reason about all types of spatial and temporal interactions.
- 4) In simulation benchmarks with dense crowds and dense obstacles, our method outperforms previous state-of-the-art methods in unseen obstacle layouts. In addition, our method demonstrates better generalization w.r.t. different human and obstacle densities.
- 5) We successfully transfer the robot policy learned in a low-fidelity simulator to challenging real-world,

everyday dynamic environments such as narrow corridors and densely packed public spaces.

II. RELATED WORKS

In this section, we first review the literature of robot crowd navigation (Sec. II-A and II-B). Then, we narrow down to crowd navigation efforts in constrained spaces (Sec. II-C). Finally, we review graph attention mechanism with a focus on its usage in multi-agent interaction modeling (Sec. II-D).

A. Model-Based Methods

Robot navigation in human crowds is particularly challenging and has been studied for decades [4], [10], [11], [12]. Model-based approaches have explored various mathematical models to optimize robot actions [4], [5], [13], [14], [15], [16]. As an early example, ROS navigation stack [17] uses a cost map for global planning and dynamic window approach (DWA) for local planning [4]. DWA searches for the optimal velocity that brings the robot to the goal with the maximum clearance from any obstacle while obeying the dynamics of the robot. By treating humans as obstacles, DWA exhibits myopic behaviors such as oscillatory paths in dynamic environments [18]. As a step forward, optimal reciprocal collision avoidance (ORCA) and social force account for the velocities of agents. ORCA models other agents as velocity obstacles and assumes that agents avoid each other under the reciprocal rule [5], [13]. SF models the interactions between the robot and other agents using attractive and repulsive forces [14]. However, the hyperparameters of the model-based approaches are sensitive to crowd behaviors and thus need to be tuned carefully to ensure good performance [18], [19]. In addition, model-based methods are prone to failures, such as the freezing problem, if the assumptions such as the reciprocal rule are broken [20], [21]. In contrast, while our method also models these interactions, we learn the hyperparameters of the model from trial and error with minimal assumptions on human behaviors posed by model-based methods.

B. Learning-Based Methods

Learning-based approaches have been widely used for navigation in dynamic environments to reduce hyperparameter tuning efforts and the number of assumptions. One example is supervised learning from expert demonstrations of desired

behaviors, where the expert ranges from model-based policies [22], [23], [24], human teleoperators in simulators [25], to real pedestrians [26], [27]. Supervised learning does not require explorations of the state and action spaces of the robot, yet the performance of learned policy is limited by the quality of expert demonstrations.

Another line of work takes advantage of crowd simulators and learns policies with RL. Through trial and error, RL has the potential to learn robot behaviors that outperform model-based approaches and expert demonstrations [28]. For example, Deep V-Learning first uses supervised learning with ORCA as the expert and then uses RL to learn a value function for path planning [1], [29], [30], [31], [32]. However, Deep V-Learning assumes that state transitions of all agents are known without uncertainty. In addition, since the networks are pre-trained with supervised learning, they share the same disadvantages with OCRA, which are hard to correct by RL [8]. To address these problems, more recent efforts leverage model-free RL without supervised learning or assumptions on state transitions [8], [21], [32], [33]. Since the state transition probability of humans is uncertain, directly learning a policy network without explicitly modeling state transitions is more suitable for navigation [34]. However, the aforementioned RL works have at least one of the following two problems: (1) They focus on navigation in open spaces and isolate agents from static constraints, posing difficulties when deploying robots in the real-world; (2) They ignore all or part of interactions among entities in the scene, which are important for robot navigation in dense crowds and highly constrained environments. We discuss how prior works address the above two problems in Sec. II-C and Sec. II-D respectively.

C. Scene Representation for Constrained Environments

To deal with both static constraints and humans, some methods such as DWA [4] and DS-RNN [8], [35] use groups of small circles to represent the contours of both static obstacles and humans. While straightforward, groups of circle representation has the following problems: 1. Humans and obstacles are treated in the same way but have different effects on robot decision making. This design choice leads to suboptimal policy as we will show in Sec. V and VI. 2. Since the length of input vectors is proportional to the number of obstacles and humans, the hyperparameters of navigation algorithms easily overfit to a specific environment setting, causing performance degradation when the densities of obstacles and humans change.

Other learning-based approaches use raw sensor images or point clouds to represent the whole environment [6], [25], [34], [36], [37]. These end-to-end (e2e) pipelines have made promising progress in simulation. However, generalizing these e2e methods to real-world scenarios requires high-fidelity simulators [38], [39], [40] and/or datasets [26], [41], [42]. Since accurate pedestrian simulation is technically difficult and expensive [40], learning a deployable e2e policy in dense and interactive crowds remains an open challenge. To circumvent this problem, other works leverage processed state information such as occupancy maps [28], [43]. However, compared to human circles or point clouds, occupancy maps expand the state space, which reduces training efficiency. In addition, the

discretization of agent positions limits the ability of occupancy maps to capture slow agent movements. For this reason, we develop a structured input representation, consisting of human circles and processed obstacle point cloud, that only includes the most essential information for navigation. This input representation leads to strong performance in both simulation and real-world while only requiring a low-fidelity simulator.

D. Graph Attention Networks for Multi-Agent Interactions

In recent years, attention mechanisms have demonstrated success in various fields [44], [45], [46]. Vaswani et al. propose a transformer with self-attention mechanism that achieves state-of-the-art performance in machine translation [44]. Later, graph attention networks show the effectiveness of attention on learning relationships of data with graphical structures such as citation networks [45]. Inspired by these works, researchers in trajectory prediction and crowd navigation have found that attention networks are also well-suited to capture interactions amongst agents and entities, which contain essential information for multi-agent tasks [1], [8], [46], [47], [48]. For each agent, these works compute attention scores for all neighboring agents. Due to the permutation invariance property, attention scores better capture the importance of pair-wise relationships than encoding the states of all agents with a concatenation operation or an recurrent network [29], [30], [35].

More specifically, a line of works uses a robot-human attention network to determine the relative importance of each human to the robot [1], [8], [49], [50]. However, interactions among humans, which can also influence the robot, are not explicitly modeled. To this end, another line of works use a homogenous graph network to include both RH and HH interactions [7], [32], [51]. However, since these works feed RH and HH features to a single attention network, the resulting robot policy has difficulty reasoning the specificity of each type of feature, which limits the robot's ability to adapt to different interactions, as demonstrated in [9] and our experiments in Sec. V. To deal with this issue, some recent works use multiple attention networks to encode RH and HH features separately, leading to improved navigation performance in dense and interactive crowds [2], [3], [32]. However, these works only deal with open-world social navigation and thus ignore the interactions between agents and obstacles, which are common in real-world navigation scenarios.

To handle obstacles, Liu et al. [52] and Chen et al. [9] treat the humans, the robot, and static objects as nodes in graph neural networks to model the spatio-temporal interactions among all observed entities. Different from Chen et al. [9] where a robot needs to reach an object such as a chair, the robot in this work only needs to avoid all objects. Thus, Chen et al. [9] represent static obstacles as a semantic map with a class label for each object, whereas we represent all observed static obstacles as a point cloud. In Liu et al. [52], representing each obstacle as a geometric shape requires further map processing, such as obstacle segmentation. Thus, we treat the point cloud consisting of all obstacles as a single node in our st-graph, which reduces both the processing requirement and the size of the graph.

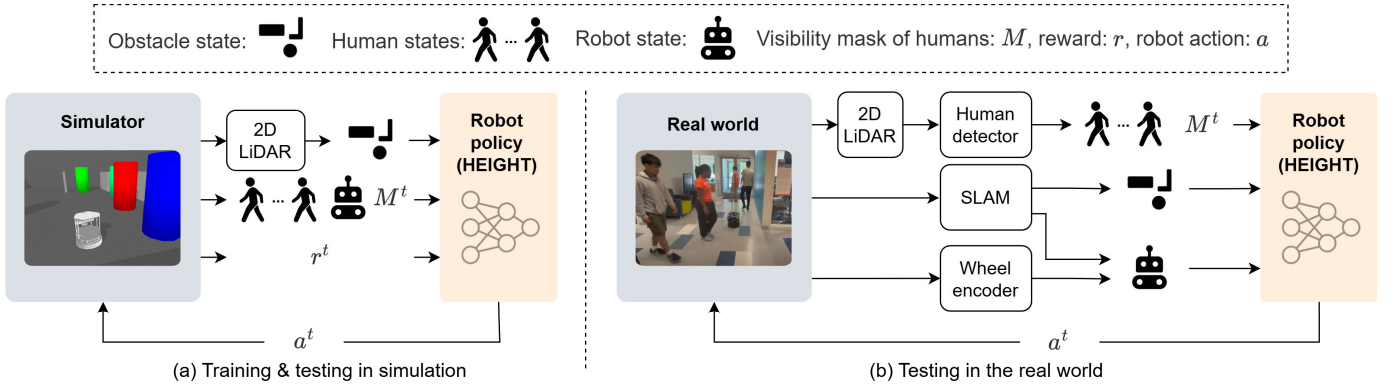


Fig. 2. **An overview of our pipeline in simulation and real-world.** (a) At each timestep t in training and testing, the simulator provides a reward r^t and the following observations of the environment: obstacle point cloud o^t , the robot state w^t , and the human states $h_1^t, \dots, h_n^t$, and masks M^t (Sec. IV-B1). These observations serve as inputs to HEIGHT, which outputs a robot action a^t that maximizes the future expected return R^t . The simulator executes the actions of all agents and the loop continues. (b) The testing loop in the real-world is similar to the simulator except the perception modules for obtaining the observations are different and the reward is absent.

III. PRELIMINARIES

In this section, we present our problem formulation, scene representation, and reward function.

A. Problem Formulation

We model the constrained crowd navigation scenario as a Markov Decision Process (MDP), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma, S_0 \rangle$. Let w^t be the robot state which consists of the robot's position (p_x, p_y) , instantaneous velocity (u_x, u_y) , goal position (g_x, g_y) , and heading angle θ in world frame. Let h_i^t be the current state of the i -th detected human at time t , which consists of the human's relative position w.r.t. the robot and its instantaneous velocity $(p_x^i - p_x, p_y^i - p_y, u_x^i, u_y^i)$ in world frame in simulation experiments. In real world experiments, the state of the i -th human only includes its relative position $h_i^t = (p_x^i - p_x, p_y^i - p_y)$ since accurate estimation of human velocities are difficult to obtain. Let o^t be the current observation of the static obstacles and walls, which is represented as a 2D point cloud. We define the state $s^t \in \mathcal{S}$ of the MDP as $s^t = [w^t, o^t, h_1^t, \dots, h_n^t]$ if a total number of n humans are detected at the timestep t , where n may change within a range in different timesteps. The IDs of humans are untracked, and their states are ordered by an increasing L_2 distance to the robot. We add a small Gaussian noise to the ground truth positions and velocities of all agents to compensate for perception errors. The state space $\mathcal{S}$ is continuous and is expanded in Sec. III-B.

In each episode, the robot begins at an initial state $s^0 \in \mathcal{S}_0$. As shown in Fig. 2(a), according to its policy $\pi(a^t|s^t)$, the robot takes an action $a^t \in \mathcal{A}$ at each timestep t . The action of the robot consists of the desired translational and rotational accelerations $a^t = [a_{trans}^t, a_{rot}^t]$. The action space $\mathcal{A}$ is discrete: the translational acceleration $a_{trans} \in \{-0.05 \text{ m/s}^2, 0 \text{ m/s}^2, 0.05 \text{ m/s}^2\}$ and the rotational acceleration $a_{rot} \in \{-0.1 \text{ rad/s}^2, 0 \text{ rad/s}^2, 0.1 \text{ rad/s}^2\}$. The robot translational velocity is $v_{trans}^t = v_{trans}^{t-1} + a_{trans}^t \times \Delta t \in [-0.5 \text{ m/s}, 0.5 \text{ m/s}]$ and rotational velocity is $v_{rot}^t = v_{rot}^{t-1} + a_{rot}^t \times \Delta t \in [-1 \text{ rad/s}, 1 \text{ rad/s}]$. The robot motion is governed by differential drive dynamics. In return, the robot receives a

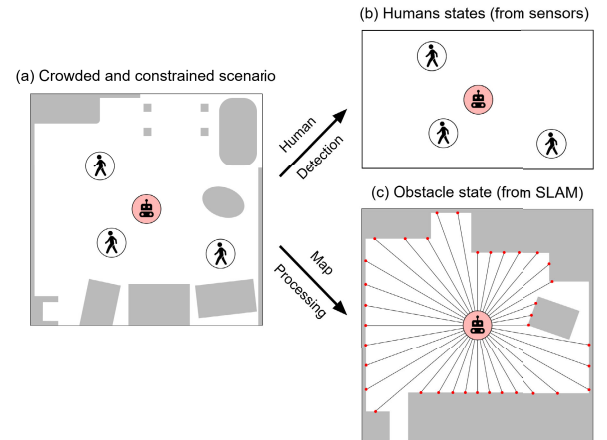


Fig. 3. **A split representation of a crowded and constrained navigation scenario.** In (b), individual human states are represented by circles. To represent static obstacles, we preprocess and smooth the original map (a). From this processed map and the robot's localization, we generate an artificial point cloud (c) that simulates obstacle perception.

reward r^t (see Sec. III-C for details) and transits to the next state s^{t+1} according to an unknown state transition $\mathcal{P}(\cdot|s^t, a^t)$. Meanwhile, all other humans also take actions according to their policies and move to the next states with unknown state transition probabilities. The process continues until the robot reaches its goal, t exceeds the maximum episode length $T = 491$ steps, or the robot collides with any humans or static obstacles.

The goal of the robot is to maximize the expected return, $R^t = \mathbb{E} \left[\sum_{i=t}^T \gamma^{i-t} r^i \right]$, where γ is a discount factor. The value function $V^\pi(s)$ is defined as the expected return starting from s , and successively following policy π .

B. Scene Representation

To aid policy learning with the complicated state space involving multiple agents and entities, we develop a structured representation that splits human and obstacle states to account for their different effects on robot decision making. In Fig. 3,

at each timestep t , we split a scene into a human representation $h_1^t, \dots, h_n^t$, and an obstacle representation o^t . Each human has its own state since it can move independently from one another, whereas all obstacles are mixed together since they are all static. In human representation (Fig. 3(b)), the position and velocity of each human are detected using off-the-shelf human detectors [53], [54], [55]. Tracking each human is not required, further simplifying perception requirements. By representing each human as a low-dimensional state vector, we abstract away detailed information such as gaits and appearance, which are difficult to simulate accurately [38], [56].

We use a 2D point cloud as the obstacle representation, which offers the following two variants. The first variant uses a real-time 2D point cloud obtained from onboard sensors. The second variant constructs an “artificial” point cloud which is computed from a pre-built map and the robot’s localization in the map. In this case, the point cloud is generated by raycasting from the robot’s estimated pose on the map, representing only static obstacles and unaffected by humans. The first variant reflects the robot’s instantaneous perception of its surroundings but requires a high-fidelity simulation of LiDAR data. The second variant is more robust to occlusions, imperfect human models, and sensor noise, but assumes the availability of a map and reliable localization. In this work, since we have access to reliable robot localization and a low-fidelity simulator with inaccurate obstacle shapes and LiDAR point clouds, we opt for the second choice to reduce sim2real gap and improve the policy’s real-world performance.

C. Reward Function

Our reward function consists of three parts. The first and main part of the function awards the robot for reaching its goal and penalizes the robot for colliding with or getting too close to humans or obstacles. In addition, we add a potential-based reward to guide the robot to approach the goal:

$$r_{main}(s^t, a^t) = \begin{cases} 20, & \text{if } d_{goal}^t \leq \rho_{robot} \\ -20, & \text{else if } d_{min}^t \leq 0 \\ d_{min}^t - 0.25, & \text{else if } 0 < d_{min}^t < 0.25 \\ 4(d_{goal}^{t-1} - d_{goal}^t), & \text{otherwise.} \end{cases} \quad (1)$$

where ρ_{robot} is the radius of the robot, d_{goal}^t is the $L2$ distance between the robot and its goal at time t , and d_{min}^t is the minimum distance between the robot and any human or obstacle at time t . The second part is a spinning penalty r_{spin} to penalize high rotational velocity:

$$r_{spin}(s^t, a^t) = -0.05 \|\omega^t\|_2^2 \quad (2)$$

where ω_t is the current rotational velocity of the robot. Finally, we add another small constant penalty $r_{time} = -0.025$ at each timestep to encourage the robot to finish the episode as soon as possible.

The robot’s reward function is the sum of the three parts:

$$r(s^t, a^t) = r_{main}(s^t, a^t) + r_{spin}(s^t, a^t) + r_{time} \quad (3)$$

Intuitively, the robot gets a high reward when it approaches the goal with a high speed and a short and smooth path, while

maintaining a safe distance from dynamic and static obstacles. The coefficient of each reward term indicates the priority of each subtask (e.x. reaching the goal and avoiding collisions are more important than others) and can be tuned if the desired priority changes.

IV. METHODOLOGY

In this section, we present our approach to decompose the constrained crowd navigation scenario as a heterogeneous st-graph, which leads to the derivation of the HEIGHT architecture in a structured manner.

A. Heterogeneous Spatio-Temporal Graph

The subtle yet highly dynamic interactions among agents and entities are important factors that makes crowd navigation difficult. To model these interactions in a structured manner, we formulate the navigation scenario as a heterogeneous st-graph. In Fig. 4a, at each timestep t , our heterogeneous st-graph $\mathcal{G}^t = (\mathcal{V}^t, \mathcal{E}^t)$ consists of a set of nodes $\mathcal{V}^t$ and a set of edges $\mathcal{E}^t$. The nodes include the detected humans $h_1^t, \dots, h_n^t$ and the robot w^t . In addition, an obstacle node o^t represents the point cloud of all obstacles as a whole. At each timestep t , the spatial edges that connect different nodes denote the spatial interactions among nodes. Different spatial interactions have different effects on robot decision-making. Specifically, since we have control of the robot but not the humans, RH interactions have direct effects while HH interactions have indirect effects on the robot actions. As an example of indirect effects, if human A aggressively forces human B to turn toward the robot’s front, the robot has to respond as a result of the interaction between A and B. Additionally, since the agents are dynamic but the obstacles are static, interactions among agents are mutual while the influence of static obstacles on agents is one-way. Thus, we categorize the spatial edges into three types: HH edges (blue in Fig. 4), obstacle-agent (OA) edges (orange), and RH edges (red). The three types of edges allow us to factorize the spatial interactions into HH, OA, and RH functions. Each function is parameterized by a neural network that has learnable parameters. Compared to previous works that ignore some edges [1], [2], [8], our method allows the robot to reason about all observed spatial interactions that exist in constrained and crowded environments.

Since the movements of all agents cause the visibility of each human to change dynamically, the set of nodes $\mathcal{V}^t$ and edges $\mathcal{E}^t$ and the parameters of the interaction functions may change correspondingly. To this end, we integrate the temporal correlations of the graph $\mathcal{G}^t$ at different timesteps using another function denoted by the purple box in Fig. 4(a). The temporal function connects the graphs at adjacent timesteps, which overcomes the short-sightedness of reactive policies and enables long-term decision-making of the robot.

To reduce the number of parameters, the same type of edges in Fig. 4(a) share the same function parameters. This parameter sharing is important for the scalability of our graph because the number of parameters is kept constant when the number of human changes [57].

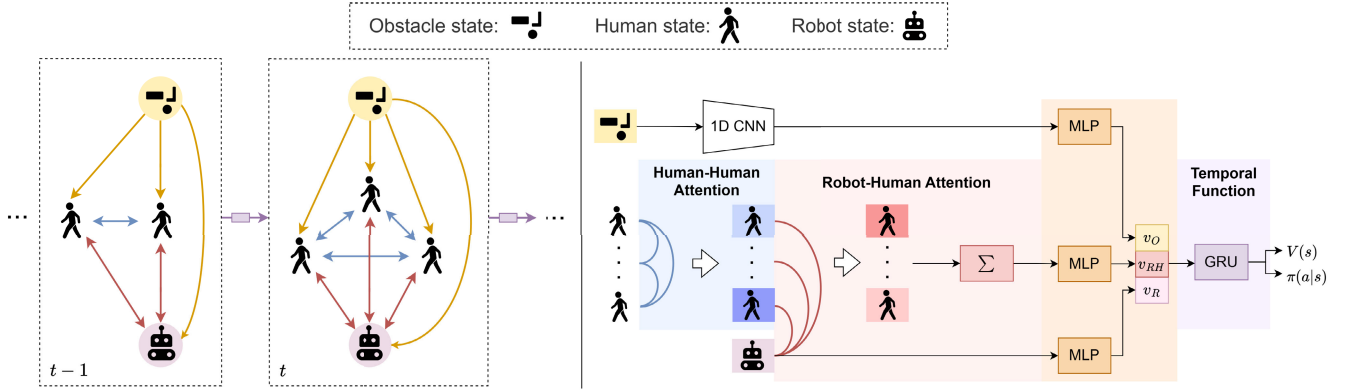


Fig. 4. **The heterogeneous st-graph and the HEIGHT network architecture.** (a) Graph representation of crowd navigation. The robot node is w (pink), the i -th human node is h_i (white), and the obstacle node is o (yellow). HH edges and HH functions are in blue, OA edges and OA functions are in orange, and RH edges and RH functions are in red. The temporal function is in purple. (b) HEIGHT network. Two attention mechanisms are used to model the HH and RH interactions. We use MLPs and a concatenation for obstacle-agent interactions, and a GRU for the temporal function. The superscript t that indicates the timestep and the human mask M is eliminated for clarity.

B. Height Architecture

In Fig. 4b, we derive our network architecture from the heterogeneous st-graph. We represent the HH and RH functions as feedforward networks with attention mechanisms, referred to as HH attn and RH attn respectively. We represent the OA function as an MLP with concatenation, and the temporal function as a gated recurrent unit (GRU). We use W and f to denote trainable weights and fully connected layers.

1) *Attention Among Agents:* The attention modules assign weights to all edges that connect to a robot or a human node, allowing the node to pay attention to important edges or interactions. The two attention networks are similar to the scaled dot-product attention with a padding mask [44], which computes the attention score using a query Q and a key K , and applies the normalized score to a value V , which results in a weighted value v .

$$v := \text{Attn}(Q, K, V, M) = \text{softmax}\left(\frac{(QK^\top + M)}{\sqrt{d}}\right) V \quad (4)$$

where d is the dimension of the queries and keys and acts as a scaling factor. The mask M is used to handle the changing number of detected humans at each timestep, as we will expand below.

Human-human attention: To learn the importance of each HH edge to the robot decision at time t , we first weigh each observed human w.r.t. other humans using an HH attention network, which is a self-attention among humans. In HH attention, the current states of humans $\mathbf{h}_1^t, \dots, \mathbf{h}_n^t$ are concatenated and passed through linear layers with weights W_{HH}^Q , W_{HH}^K , and W_{HH}^V to obtain $Q_{HH}^t, K_{HH}^t, V_{HH}^t \in \mathbb{R}^{n \times d_{HH}}$, where d_{HH} is the attention size for the HH attention.

$$\begin{aligned} Q_{HH}^t &= [\mathbf{h}_1^t, \dots, \mathbf{h}_n^t]^\top W_{HH}^Q = [q_1, \dots, q_n]^\top \\ K_{HH}^t &= [\mathbf{h}_1^t, \dots, \mathbf{h}_n^t]^\top W_{HH}^K = [k_1, \dots, k_n]^\top \\ V_{HH}^t &= [\mathbf{h}_1^t, \dots, \mathbf{h}_n^t]^\top W_{HH}^V = [v_1, \dots, v_n]^\top \end{aligned} \quad (5)$$

where $q_i \in \mathbb{R}^{1 \times d_{HH}}$, $k_i \in \mathbb{R}^{1 \times d_{HH}}$, and $v_i \in \mathbb{R}^{1 \times d_{HH}}$ are the query embedding, key embedding, and value embedding of the i -th human, respectively.

In addition, following Eq. 4, a mask $M^t \in \mathbb{R}^{n \times n}$ indicates the visibility of each human to the robot at current time t and masks out humans that are undetectable by the robot sensors. Assume the n -th human is not visible at time t . Then M^t is a matrix filled with zeros, except that every entry in n -th column is $-\infty$. The numerator of Eq. 4 can be express as

$$\begin{aligned} & Q_{HH}^t \cdot (K_{HH}^t)^\top + M^t \\ &= \begin{bmatrix} q_1 k_1 & \dots & q_1 k_{n-1} & q_1 k_n \\ \vdots & \ddots & \vdots & \vdots \\ q_n k_1 & \dots & q_n k_{n-1} & q_n k_n \end{bmatrix} + \begin{bmatrix} 0 & \dots & 0 & -\infty \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & -\infty \end{bmatrix} \\ &= \begin{bmatrix} q_1 k_1 & \dots & q_1 k_{n-1} & -\infty \\ \vdots & \ddots & \vdots & \vdots \\ q_n k_1 & \dots & q_n k_{n-1} & -\infty \end{bmatrix} \end{aligned} \quad (6)$$

Let $V_{HH}^t = [v_1, \dots, v_n]^\top$, where $v_i \in \mathbb{R}^{1 \times d_{HH}}$ is the value embedding of the i -th human. Then, based on Eq. 4, the weighted human embeddings $v_{HH}^t \in \mathbb{R}^{n \times d_{HH}}$ is

$$\begin{aligned} v_{HH}^t &= \text{softmax}\left(\frac{Q_{HH}^t (K_{HH}^t)^\top + M^t}{\sqrt{d}}\right) \cdot V_{HH}^t \\ &= \begin{bmatrix} c_1 q_1 k_1 & \dots & c_1 q_1 k_{n-1} & 0 \\ \vdots & \ddots & \vdots & \vdots \\ c_n q_n k_1 & \dots & c_n q_n k_{n-1} & 0 \end{bmatrix} \cdot \begin{bmatrix} v_1 \\ \vdots \\ v_{n-1} \\ v_n \end{bmatrix} \\ &= \begin{bmatrix} c_1 q_1 k_1 v_1 + \dots + c_1 q_1 k_{n-1} v_{n-1} \\ \vdots \\ c_n q_n k_1 v_1 + \dots + c_n q_n k_{n-1} v_{n-1} \end{bmatrix} \end{aligned} \quad (7)$$

where $c_1, \dots, c_n$ are constants that reflect the effect of the scaling factor d and the softmax function. Thus, the value of the n -th missing human v_n is eliminated by the mask M^t and thus does not affect the resulting weighted human embedding v_{HH}^t . The mask that indicates the visibility of each human prevents attention to undetected humans, which is common in crowd navigation due to the partial observability caused by the limited robot sensor range, occlusions, imperfect human detectors, etc [33]. Additionally, the mask provides unbiased

gradients to the networks, which stabilizes and accelerates the training [2], [58].

Robot-human attention: After the humans are weighted by HH attention, we weigh their embeddings again w.r.t. the robot with another RH attention network to learn the importance of each RH edge. In RH attention, we first embed the robot state w^t with a fully connected layer, which results in the key for RH attention $K_{RH}^t \in \mathbb{R}^{1 \times d_{RH}}$. The query and the value, $Q_{RH}^t, V_{RH}^t \in \mathbb{R}^{n \times d_{RH}}$, are the other two linear embeddings of the weighted human features from HH attention v_{HH}^t .

$$Q_{RH}^t = v_{HH}^t W_{RH}^Q, K_{RH}^t = w^t W_{RH}^K, V_{RH}^t = v_{HH}^t W_{RH}^V \quad (8)$$

We compute the attention score from $Q_{RH}^t, K_{RH}^t, V_{RH}^t$, and the mask M^t to obtain the weighted human features for the second time $v_{RH}^t \in \mathbb{R}^{1 \times d_{RH}}$ as in Eq. 4.

2) *Incorporating Obstacle and Temporal Information:* We first feed the point cloud that represents obstacles, o^t , into a 1D-CNN followed by a fully connected layer to get an obstacle embedding v_O^t . Then, we embed the robot states w^t with linear layers f_R to obtain a robot embedding v_R^t .

$$v_O^t = f_{CNN}(o^t), \quad v_R^t = f_R(w^t) \quad (9)$$

Finally, the robot and obstacle embeddings are concatenated with the twice weighted human features v_{RH}^t and fed into the GRU¹. Thus, we keep the simple concatenation design to incorporate obstacle information.:

$$h^t = \text{GRU}(h^{t-1}, ([v_{RH}^t, v_R^t, v_O^t])) \quad (10)$$

where h^t is the hidden state of GRU at time t . Finally, the h^t is input to a fully connected layer to obtain the value $V(s^t)$ and the policy $\pi(a^t|s^t)$.

3) *Training:* We train the entire network with Proximal Policy Optimization (PPO) [59] in a simulator as shown in Fig. 2(a) (see Sec. V-A for details of the simulator). At each timestep t , the simulator provides all state information that constitutes s^t , which is fed to the HEIGHT network. The network outputs the estimated value of the state $V(s^t)$ and the logarithmic probabilities of the robot's action $\pi(a^t|s^t)$, both of which are used to compute the PPO loss and then update the parameters in the network. In training, the robot action is sampled from the action distribution $\pi(a^t|s^t)$. In testing, the robot takes the action with the highest probability a^t . The robot action a^t is fed into the simulator to compute the next state s^{t+1} , and then the loop continues.

Without any supervised learning, our method is not limited by the performance of expert demonstrations [1], [29], [51]. However, to improve the low training data efficiency, an inherent problem of RL, HEIGHT can also be trained with a combination of imitation learning and RL.

¹From experiments, we find that adding a third obstacle-agent attention network, where the obstacle embedding is the key and the robot and human embeddings are the query and value, does not improve the performance. The reason is that the agents only interact with a small subset of obstacles instead of the whole point cloud at each timestep, eliminating the need for modeling these interactions.

4) *Summary:* We present a structured and principled approach to design the robot policy network for crowd navigation in constrained environments. By decomposing the complex scenario into independent components, we split the complex problem into smaller functions, which are used to learn the parameters of the corresponding functions. By combining all components above, the end-to-end trainable HEIGHT allows the robot to perform spatial and temporal reasoning on all pairwise interactions, leading to better navigation performance.

V. SIMULATION EXPERIMENTS

In this section, we present our environment, experiment setup, and results in simulation. Our experiments are guided by the following questions: (1) What is the advantage of our split scene representation compared with alternative representations? (2) What is the importance of the graph formulation and why do we differentiate types of edges with a heterogeneous st-graph? (3) What is the importance of HH and RH attention in HEIGHT? (4) What are the failure cases of our method?

A. Simulation Benchmark

We conduct simulation experiments in a PyBullet simulator [60]. The robot, humans, and static obstacles are in a 12m×12m arena. In each episode, rectangular obstacles are initialized with random shapes and random poses. The width and the length of each static obstacle are sampled from $\mathcal{N}(1, 0.6^2)$ and are limited in [0.1, 5] meters. The initial positions of the humans and the robot are also randomized. The starting and goal positions of the robot are randomly sampled inside the arena. The distance between the robot's starting and the goal positions is between 5 m and 6 m. Some humans are dynamic and some are static. The goals of dynamic humans are set on the opposite side of their initial positions to create circle-crossing scenarios. In training, the number of humans varies from 5 to 9 and the number of static rectangular obstacles varies from 8 to 12. Among all humans, 0-2 of them are static and the rest are dynamic. In testing, the number of humans and obstacles are shown in the first column of Table II. The heights of humans and obstacles are all above the height of the robot LiDAR to ensure detectability.

To simulate a continuous human flow similar to [2], [8], and [28], dynamic humans will move to new random goals immediately after they arrive at their goal positions or they get stuck in front of narrow passageways for more than 10 timesteps. All dynamic humans are controlled by ORCA [13]. 80% of dynamic humans do not react to the robot and 20% of humans react to the robot. This mixed setting prevents our network from learning an extremely aggressive policy in which the robot forces all humans to yield while achieving a high reward. Meanwhile, the simulation maintains enough reactive humans to resemble the real crowd behaviors. We use holonomic kinematics for humans. The preferred speed of humans ranges from 0.4 m/s to 0.6 m/s to accommodate the speed of the robot. We assume that humans can achieve the desired velocities immediately, and they will keep moving with these velocities for the next Δt seconds. The simulation

TABLE I
COMPARISON OF SCENE REPRESENTATION AND INTERACTION MODELING OF ALL METHODS

Method	Scene Representation (humans and static obstacles)	Navigation algorithm
DWA [4]	Circles for both	Treats humans as obstacles, no interaction modeling
ORCA [13]	Circles for humans + Line segments for obstacles	Optimization program with velocity obstacles to model all interactions
A*+CNN [6]	LiDAR point cloud for both	A* + RL policy with no interaction modeling
DS-RNN [8]	Circles for both	RL policy with only RH attention and GRU
DRL-VO [28]	Occupancy map for humans + Point cloud for both	Pure pursuit algorithm + RL policy with no interaction modeling
HomoGAT [45]	Circles for humans + Point cloud for obstacles	RL policy from st-graph with shared attention weights for all interactions
Ours	Circles for humans + Point cloud for obstacles	RL policy from st-graph with separate functions for all interactions

TABLE II
BASELINE COMPARISON RESULTS WITH DIFFERENT HUMAN AND OBSTACLE DENSITIES IN UNSEEN ENVIRONMENTS

Environment	Method	Success $\uparrow$	Collision $\downarrow$			Timeout $\downarrow$	Nav Time $\downarrow$	Path Len $\downarrow$
			Overall	w/ Humans	w/ Obstacles			
Training distribution 5-9 humans 8-12 obstacles	DWA [4]	0.16	0.68	0.63	0.05	0.15	28.45	11.52
	ORCA [13]	0.34	0.62	0.31	0.32	0.04	21.51	9.46
	A*+CNN [6]	0.64	0.29	0.28	0.01	0.07	25.72	12.30
	DS-RNN [8]	0.80	0.20	0.11	0.09	0.00	19.90	10.78
	DRL-VO [28]	0.59	0.41	0.34	0.07	0.00	21.45	10.36
	HomoGAT [45]	0.86	0.14	0.13	0.01	0.00	18.66	10.36
	HEIGHT (ours)	0.88	0.12	0.09	0.03	0.00	18.31	10.34
Less crowded 0-4 humans 8-12 obstacles	DWA [4]	0.29	0.37	0.28	0.09	0.34	25.74	14.42
	ORCA [13]	0.42	0.54	0.11	0.42	0.04	19.77	9.69
	A*+CNN [6]	0.84	0.12	0.11	0.01	0.04	22.64	11.73
	DS-RNN [8]	0.72	0.28	0.07	0.21	0.00	17.21	9.56
	DRL-VO [28]	0.82	0.17	0.12	0.05	0.01	19.14	10.53
	HomoGAT [45]	0.95	0.05	0.03	0.02	0.00	16.64	10.17
	HEIGHT (ours)	0.97	0.03	0.02	0.01	0.00	16.32	10.04
More crowded 10-14 humans 8-12 obstacles	DWA [4]	0.11	0.78	0.74	0.04	0.11	29.60	10.37
	ORCA [13]	0.26	0.71	0.41	0.30	0.03	23.33	9.21
	A*+CNN [6]	0.47	0.42	0.39	0.03	0.11	27.33	12.47
	DS-RNN [8]	0.76	0.22	0.16	0.06	0.01	23.08	11.67
	DRL-VO [28]	0.50	0.49	0.41	0.09	0.01	22.72	10.26
	HomoGAT [45]	0.72	0.28	0.23	0.05	0.00	20.46	10.57
	HEIGHT (ours)	0.78	0.22	0.19	0.03	0.00	19.69	10.39
Less constrained 5-9 humans 3-7 obstacles	DWA [4]	0.17	0.57	0.55	0.02	0.26	28.50	14.21
	ORCA [13]	0.42	0.54	0.30	0.24	0.03	21.89	10.59
	A*+CNN [6]	0.66	0.29	0.28	0.01	0.05	23.63	11.98
	DS-RNN [8]	0.66	0.34	0.20	0.14	0.00	18.01	9.76
	DRL-VO [28]	0.70	0.30	0.26	0.04	0.00	21.21	11.15
	HomoGAT [45]	0.88	0.12	0.10	0.02	0.00	17.66	10.37
	HEIGHT (ours)	0.90	0.10	0.09	0.01	0.00	17.20	10.23
More constrained 5-9 humans 13-17 obstacles	DWA [4]	0.14	0.66	0.49	0.17	0.20	30.47	11.35
	ORCA [13]	0.30	0.58	0.24	0.34	0.11	22.40	9.04
	A*+CNN [6]	0.48	0.29	0.23	0.06	0.23	27.28	13.11
	DS-RNN [8]	0.71	0.23	0.09	0.14	0.06	23.45	11.58
	DRL-VO [28]	0.55	0.40	0.23	0.07	0.05	21.87	10.22
	HomoGAT [45]	0.80	0.20	0.11	0.09	0.00	19.20	10.51
	HEIGHT (ours)	0.84	0.15	0.07	0.08	0.01	18.79	10.65

and control frequency Δt is 0.1 s. The maximum length of an episode is 491 timesteps or 49.1 s.

B. Setup

1) *Baselines*: We compare the performance of our method with classical and state-of-the-art baselines listed in the top 5 row of Table I. The baselines vary in terms of both scene representation and navigation algorithm. A detailed description of all baselines can be found in Appendix A.

2) *Ablations*: To show the benefits of each attention network, we perform an ablation study on the two attention models. The ablated models are defined as follows:

- **No attn**: Both RH and HH attention networks are removed. No attn model only has the embedding layers for the inputs and the GRU.
- **RH**: The HH attention network is removed and the humans are weighted only once w.r.t. the robot. Everything else is the same as ours.
- **HH**: The RH attention network is removed and the humans are weighted only once w.r.t. other humans. Everything else is the same as ours.
- **RH+HH (ours)**: The full network as shown in Fig. 4(b).

3) *Training*: For fair comparison, we train all RL methods, including all baselines and ablations except DWA, with the

TABLE III
ABLATION STUDY RESULTS WITH DIFFERENT HUMAN AND OBSTACLE DENSITIES IN UNSEEN ENVIRONMENTS

Environment	Method	Success $\uparrow$	Collision $\downarrow$			Timeout $\downarrow$	Nav Time $\downarrow$	Path Len $\downarrow$
			Overall	w/ Humans	w/ Obstacles			
Training distribution	No attn	0.52	0.46	0.41	0.05	0.02	26.48	11.81
	5-9 humans	0.85	0.15	0.13	0.02	0.00	18.86	10.41
	8-12 obstacles	0.87	0.12	0.10	0.02	0.01	18.31	10.42
	RH+HH (ours)	0.88	0.12	0.08	0.04	0.00	18.49	10.28
Less crowded	No attn	0.72	0.27	0.24	0.03	0.01	21.78	10.94
	0-4 humans	0.89	0.07	0.04	0.03	0.04	17.34	10.66
	8-12 obstacles	0.94	0.06	0.05	0.01	0.00	16.10	9.90
	RH+HH (ours)	0.97	0.03	0.02	0.01	0.00	16.32	10.04
More crowded	No attn	0.29	0.61	0.53	0.08	0.10	28.35	12.57
	10-14 humans	0.70	0.29	0.25	0.04	0.01	20.39	10.49
	8-12 obstacles	0.74	0.25	0.17	0.08	0.01	20.34	10.53
	RH+HH (ours)	0.78	0.22	0.19	0.03	0.00	19.69	10.39
Less constrained	No attn	0.55	0.43	0.42	0.01	0.01	24.64	11.75
	5-9 humans	0.86	0.14	0.12	0.02	0.00	18.17	10.33
	3-7 obstacles	0.88	0.12	0.10	0.02	0.00	17.24	10.25
	RH+HH (ours)	0.90	0.10	0.09	0.01	0.00	17.20	10.23
More constrained	No attn	0.43	0.49	0.38	0.11	0.08	26.25	11.82
	5-9 humans	0.77	0.22	0.11	0.11	0.01	20.21	10.73
	13-17 obstacles	0.84	0.16	0.06	0.10	0.00	18.94	10.66
	RH+HH (ours)	0.84	0.15	0.07	0.08	0.01	18.79	10.65

same number of timesteps and the same PPO hyperparameters. Please refer to Appendix B) for training details.

4) *Evaluation*: We test all methods with the same 500 random unseen test episodes. For RL-based methods, we test the last 5 checkpoints (equivalent to the last 6000 training steps) and report the result of the checkpoint with the highest success rate. We conduct the testing in 5 different human and obstacle densities, as shown in the first columns of Table II and Table III. The first set of densities is the same as the training environment and is thus in *training distribution*. To test the generalization of methods, we change the range of human or obstacle numbers in the remaining four environments to values that do not overlap with those used in training. Thus, these 4 environments are out-of-distribution (OOD).

All testing scenarios, including those in the *training distribution*, are unseen during training. This is because in each testing episode, a new seed that is not used in training determines the number of humans and obstacles, the starting and goal positions of all agents, and the size and pose of obstacles. Only the 4 arena walls do not change across different episodes.

The testing metrics measure the quality of the navigation and include the success rate, collision rate with humans and obstacles, timeout rate, the average navigation time of successful episodes in seconds, and path length of the successful episodes in meters.

C. Results

1) *Effectiveness of Scene Representation*: To analyze the effects of input scene representations on crowd navigation algorithms, we compare ours and HomoGAT, the two methods that distinguish human and obstacle inputs, with baselines that mix humans and obstacles in input representation: the model-based DWA, the RL-based DS-RNN, and the hybrid planner A*+CNN. The results are shown in Table II.

For DWA, treating humans as obstacles leads to the highest average human collision rates (0.54) and a freezing problem, indicated by the highest average timeout rates (0.21), in all environments. For example, the robot in Fig. 5 stays close to everything and thus fails to avoid the magenta human in time.

Similarly, by representing obstacles as groups of circles, DS-RNN performs better in the *More crowded* environment (Fig. 6(d)) than in the *More constrained* environment (Fig. 5(d)). In addition, Table II shows that DS-RNN achieves the highest average collision rate with obstacles (0.13) in all environments. Furthermore, the obstacle collision rate of DS-RNN increases in all 4 OOD environments, indicating an overfitting problem. Among the OOD environment, the percentage of obstacle collision increase is higher in environments with higher obstacle-to-human ratios. For example, in *Less crowded* with the highest percentage of obstacles, the obstacle collision rate (0.21) increases by 133% w.r.t. the obstacle collision rate in *training distribution* (0.09). On the contrary, in *More crowded* with the lowest percentage of obstacles, the obstacle collision rate (0.06) drops by 33%. The reason is that DS-RNN has trouble inferring the geometric shapes of obstacles from a large group of circles, and thus fails to avoid collision with them. Thus, treating both humans and obstacles as circles is not an ideal input representation for robot navigation algorithms.

For A*+CNN, the A* global planner does not account for humans and the CNN local planner represents all observations as a 2D point cloud. As a result, from Table II, A*+CNN has the highest average timeout rate (0.10) and the longest average time (18.99) among RL-based methods in all environments, especially the two most challenging ones, for the following 2 reasons. (1) The waypoints can fail to guide the robot to reach the goal because the waypoints lose optimality as agents move. Consequently, in OOD scenarios such as Fig. 5(b), the CNN

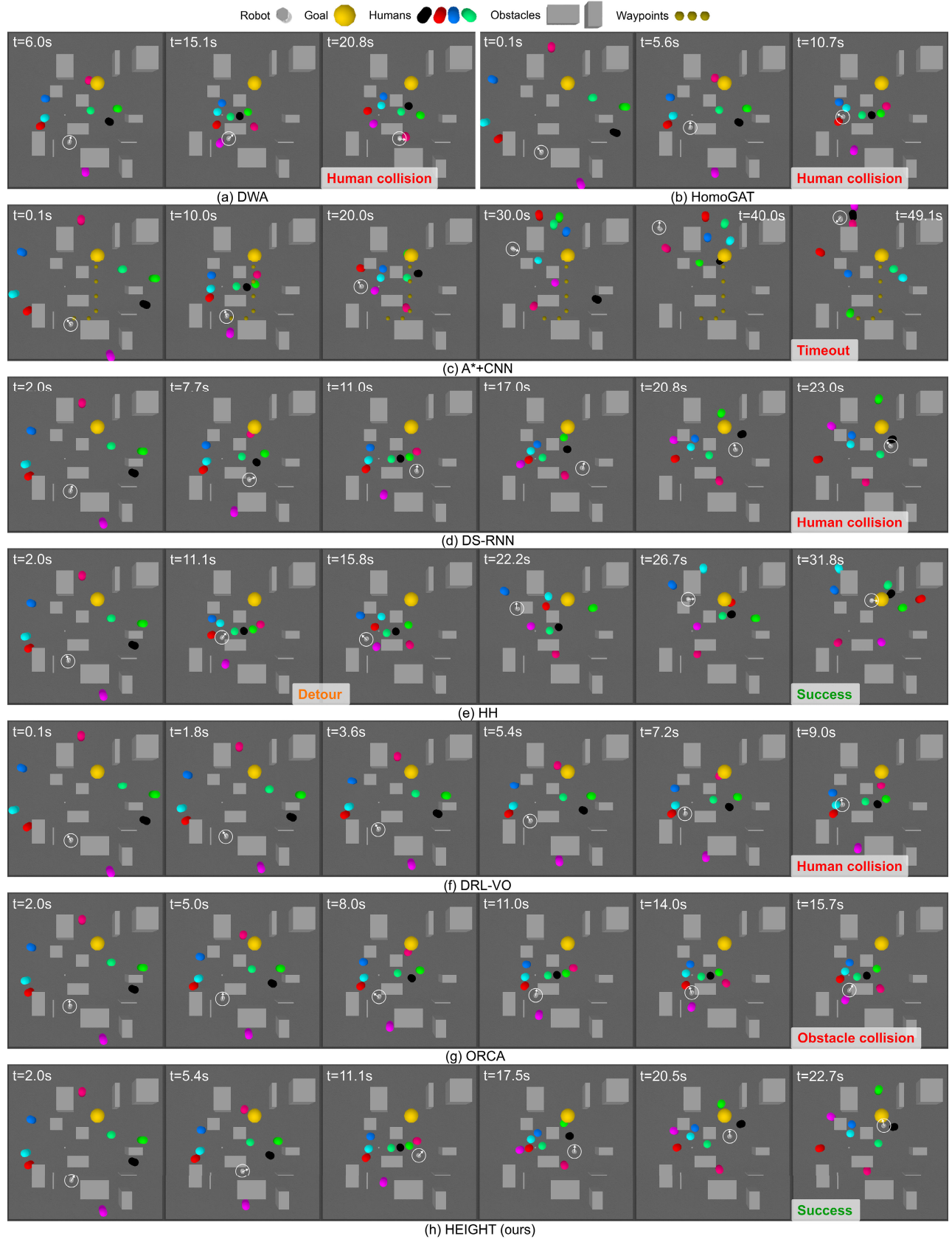


Fig. 5. Comparison of different methods in the same testing episode in *More Constrained environment*. The robot is centered in white circles and its orientation is denoted by white arrows. More qualitative results can be found in the video attachment.

policy is especially bad at long-horizon planning because it is overfitted with good waypoints. (2) By mixing humans and obstacles in its input, the robot policy sometimes has a hard time distinguishing dynamic and static obstacles. For example,

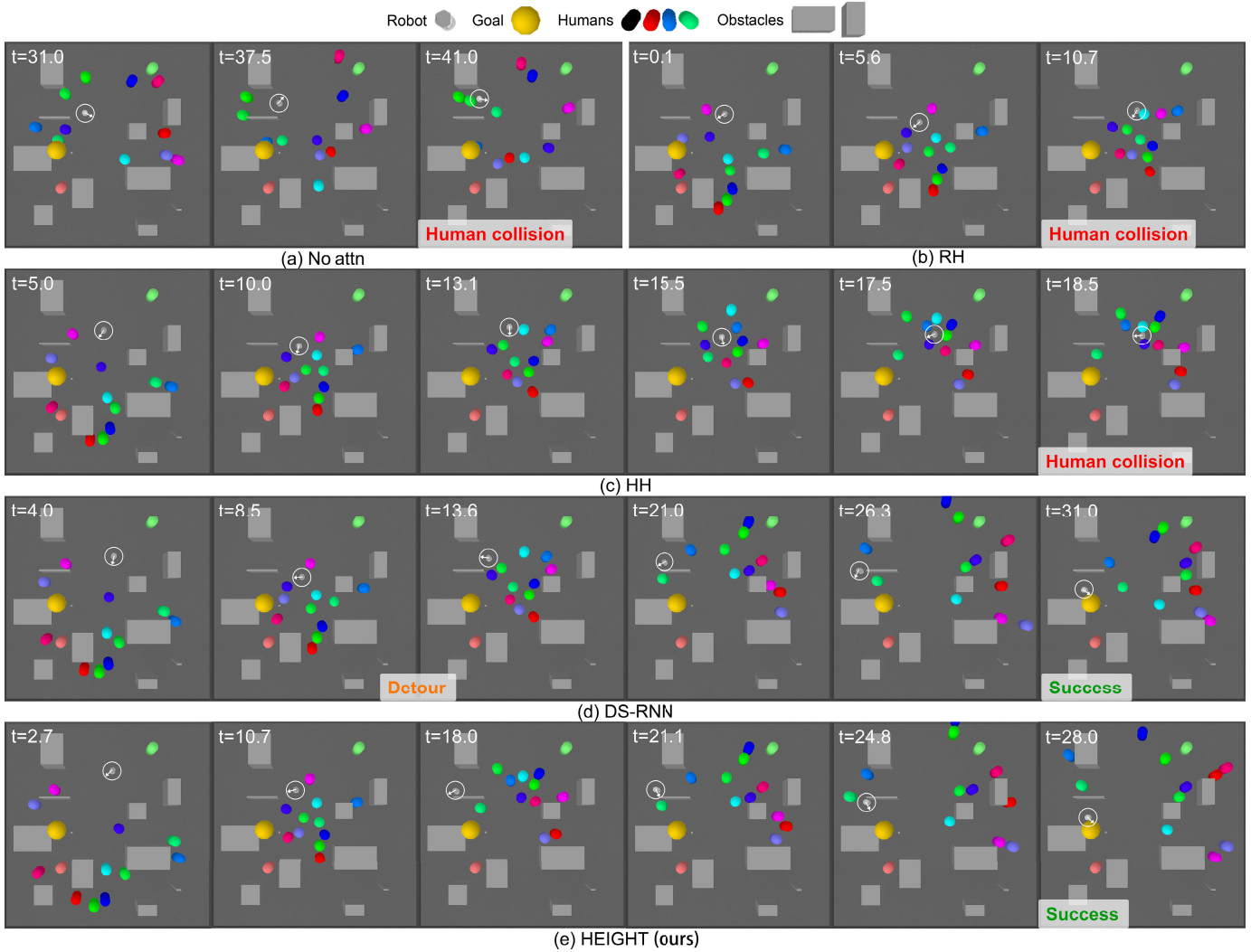


Fig. 6. Comparison of different methods in the same testing episode in *More Crowded* environment.

in Fig. 5(b), the robot keeps a large distance from everything and thus is less agile and efficient compared with the robots in Fig. 5(e) and (f). Thus, for RL-based approaches, treating humans as obstacles leads to overfitting to specific types of scenarios, preventing the policies from generalizing to OOD scenarios which are common in the real-world.

Compared with A*+CNN, the success rate of DRL-VO is not always better but degrades less in *More crowded*, *Less constrained*, and *More constrained* scenarios. This difference can be attributed to the input representation of its RL policy, which includes low-level human states encoded as occupancy maps in addition to point cloud data. Since the size of occupancy maps remains fixed regardless of the number of pedestrians, the policy generalizes well across varying human densities. However, occupancy maps have a much higher dimensionality compared to point clouds or detected human circles, which can lead to underfitting under the same training budget as HomoGAT and our method (Fig. 5(f)).

By splitting human and obstacle input representations, HomoGAT and HEIGHT achieve the top 2 performance across most metrics and environments. Especially, HEIGHT learns to

keep a larger distance from human paths yet a shorter distance from obstacles to balance safety and efficiency (Fig. 5(g)), indicated by the lowest overall collision rates (0.17) and the shortest average navigation time (18.10) in all environments. In addition, in the last 3 frames of Fig. 6(e), the robot detours around the thin and long wall because the wall will not move out of its way. However, when the robot encounters the left-most green human, it waits for the human to pass and resume afterwards because the human only blocks its way temporarily. Therefore, we conclude that our split representation consisting of detected human states and obstacle point clouds is most suitable to learn robot navigation in crowded and constrained environments with RL, among all popular choices in Table II.

2) *Effectiveness of the Heterogeneous St-Graph*: To justify our heterogeneous st-graph, we compare our method with ORCA, A*+CNN, and DRL-VO with no graph concept and HomoGAT with a homogenous graph in Table II.

ORCA models the RH and OA interactions at the current timestep as velocity obstacles. As a result, ORCA is the second worst among all methods in terms of success rate and performs especially poorly in terms of obstacle collision rates, shown

in Fig. 5(g). Since all methods that outperform ORCA contain either a search-based planner or recurrent networks, this result indicates the importance of HH interaction and long-term temporal reasoning in our problem. However, ORCA has the shortest average path length in most environments due to its greedy decisions to move toward the goal.

A*+CNN lacks structure in input representation, and subsequently, lacks structure in network architecture. For this reason, as we discussed in Sec. V-C1, A*+CNN shows much worse average success rate (0.62 v.s. 0.84), navigation time (25.32 v.s. 18.52), and path length (12.32 v.s. 10.40) than HomoGAT which has a simple st-graph in all environments. A*+CNN also exhibits larger variances in terms of success rate (0.019 v.s. 0.0060), navigation time (3.62 v.s. 1.70), and path length (0.22 v.s. 0.019) across four OOD environments. Especially in the challenging *More crowded* and *More constrained* environments, A*+CNN exhibits a larger drop in average success rate compared with HomoGAT (0.165 v.s. 0.10). A similar phenomenon can also be observed by comparing DRL-VO, which only lacks structure in network architecture, and HomoGAT. This finding shows that even a simple st-graph structure can lead to a performance gain, indicating the importance of spatial and temporal reasoning for constrained robot crowd navigation.

However, occasional failures of HomoGAT still exist such as Fig. 5(b). Without differentiating RH and OA interactions, the robot maintains similar distances from humans and obstacles, which is prone to collisions because it needs to keep larger distances from humans. As a step further, with a heterogeneous st-graph, HEIGHT treats the 3 types of edges with different network components. As a result, HEIGHT outperforms HomoGAT and achieves the best average success rate (0.87), navigation time (18.06), and path length (10.33) in all environments. HEIGHT's variance of most metrics across all testing environments are the smallest, indicating its robustness to distribution shift. The reason is that the heterogeneous components allow the robot to reason about the different effects of HH, RH, and OA spatial interactions. For example, in Fig. 5(f) and Fig. 6(e), HEIGHT chooses a path that avoids the most crowded region, yields to humans when the robot must encounter them, and stays closer to walls for efficiency. Therefore, we conclude that the spatial and temporal reasoning on different types of interactions is the key to ensuring good in-distribution performance and OOD generalization in crowded and interactive environments.

3) *Importance of Attention Networks*: We use ablations to evaluate the contribution of RH and HH attention networks to the performance of HEIGHT, as shown in Table III. A visualization of attention weights is in Fig. 10 in Appendix.

First, in terms of all metrics in most environments, the model with both RH and HH attention shows the strongest results, followed by the models with only one attention, and finally by the model with no attention. For example, in Fig. 6, the ablations in (a), (b), and (c) fail yet our full model in (e) succeeds. Especially in the 2 most challenging *More crowded* and *More constrained* environments, the existence of either RH or HH attention significantly boosts the average success rate by 0.38 and 0.43 respectively, compared with No attn. For

example, in Fig. 5(e), the robot detours and goes dangerously close to the crowds in a constrained area, yet HH attention allows it to realize the danger and resume to a less crowded path. This result shows that reasoning about both HH and RH spatial relationships is essential for our problem, especially in *more crowded* or *more constrained* environments where the spatial interactions are also dense.

Second, by comparing RH and HH, we find that HH attention plays a more important role in all environments in terms of success rate and navigation time. An extreme example can be observed from *more constrained* environment, where the success rate of RH+HH and HH are the same.

This is because the number of HH edges is larger than the number of RH edges in most cases. Thus, the robot can observe and is affected by a relatively larger number of HH interactions yet a smaller number of RH interactions.

4) *Failure Cases*: By visualizing the testing episodes across all environments, we find that HEIGHT typically collides when (1) a human arrives at its goal and suddenly changes directions to a new goal (Fig. 11(a) in Appendix) or (2) the robot surroundings are extremely crowded and constrained, and almost all free paths are blocked by humans (Fig. 11(b)). In these cases, due to its speed limit, the robot sometimes cannot switch to an alternative path in time to prevent collisions. To remedy the difficulty of RL in long-term decision making, in future work, our method can be combined with motion primitives and path planning algorithms that consider the stochasticity of pedestrian motions [61].

Besides answering the four research questions in Sec. V, we provide additional insights in Appendix C.

VI. REAL-WORLD EXPERIMENTS

In this section, we present challenges, setup, and results in everyday environments in the real-world.

A. Setup

Challenges and solutions: Real-world navigation with pedestrians and static obstacles is challenging because perception errors, unexpected human behaviors, and noisy robot dynamics model cause sim2real gaps. To deal with noises in human detection, robot localization, and robot dynamics, we inject Gaussian noises into the observed agent positions and the robot dynamics model in the simulator to robustify the learned policy. To conquer the human behavior gap, in the simulator, we design pedestrian routes based on observations of human flow and the layout of the deployment environment.

We test the policies in four everyday environments **without any additional training in the real-world**. The hallway environment (Fig. 12) has extremely narrow free space and tests the robot's ability to handle static obstacles with low density crowds. The lounge environment (Fig. 13) tests the robot's ability to avoid more pedestrians and obstacles with more diverse shapes. The atrium (Fig. 7) and the outdoor environment (Fig. 8) test the robot's potential of real-world deployment in dense and unrehearsed crowds. Details about setup can be found in Appendix D.

In the Hallway and Lounge environments, we conduct controlled experiments where the trajectories of all pedestrians

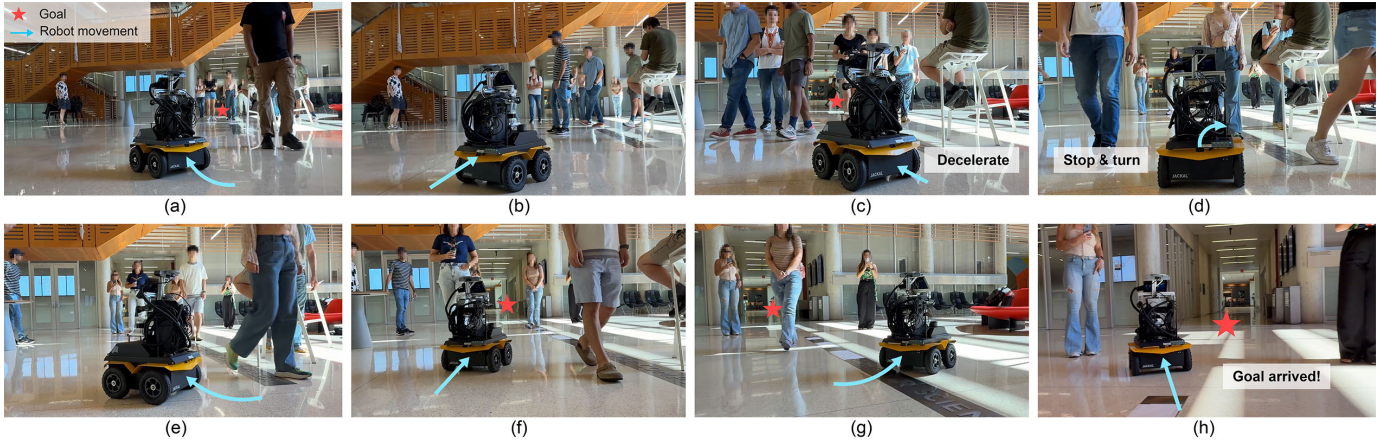


Fig. 7. A deployment scenario of our method in the Atrium environment with 14 ~ 25 humans. The jackal takes detours to avoid sparse crowds, stops and adjusts its orientation to avoid dense crowds, and arrives at the goal. More qualitative results can be found in the video attachment.

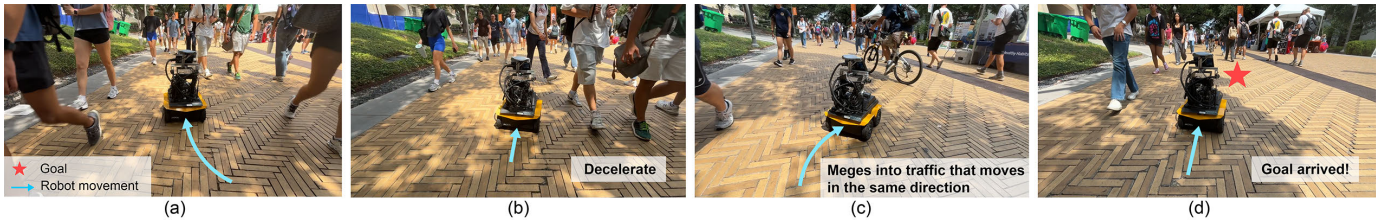


Fig. 8. A deployment scenario of our method in an outdoor environment with 35 + humans. The jackal takes detours if there is enough space, decelerates if all paths are blocked, and merges into the traffic that moves in the same direction if possible.

TABLE IV
REAL-WORLD BASELINE COMPARISON RESULTS

Environment	# of trials	Method	Success $\uparrow$	Collision $\downarrow$	Timeout $\downarrow$	Nav Time $\downarrow$
Hallway 1-2 humans	18	Navigation Stack	0.72	0.06	0.22	16.71
		DRL-VO	0.50	0.50	0.00	29.33
		HEIGHT (ours)	1.00	0.00	0.00	22.36
Lounge 1-6 humans	12	Navigation Stack	0.83	0.17	0.00	32.00
		DRL-VO	0.67	0.33	0.00	28.91
		HEIGHT (ours)	0.83	0.17	0.00	30.71

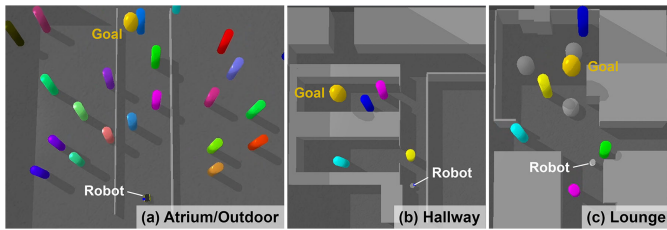


Fig. 9. Simulation environments for training sim2real policies in Sec. VI.

are fixed to ensure a fair comparison. We compare our method with the following two baselines: (1) ROS navigation stack is a geometric planner which uses the dynamic window approach (DWA) [4] as the local planner and A* as the global planner; (2) DRL-VO is a state-of-the-art RL method [28]. For each method, we run 30 trials in total. Among all trials, the start goal positions of the robot are fixed. We measure the success, collision, timeout rates, and navigation time of successful episodes as testing metrics.

To demonstrate the real-world deployment of our method, we conduct in-the-wild experiments in two densely packed public spaces: the Atrium and Outdoor environment in Fig. 7 and Fig. 8 with non-rehearsed pedestrian flows.

B. Baseline Comparison Results

In the highly constrained yet less crowded Hallway environment, ROS navigation stack often needs to spin in place to replan, as shown by the higher navigation time in Table IV. Some of the spinning recovery attempts fail and result in timeouts. In ROS navigation stack, both global and local planners treat humans as obstacles. As a result, similar to the baselines in Sec. V-C1, the robot has difficulties distinguishing dynamic obstacles that will clear out and static obstacles that will always stay, causing failures in highly constrained spaces. For the same reason as simulation experiment in Sec. V-C-1), the large input representation and network of DRL-VO pose challenges for RL exploration in this highly constrained environment. Insufficient exploration leads to an

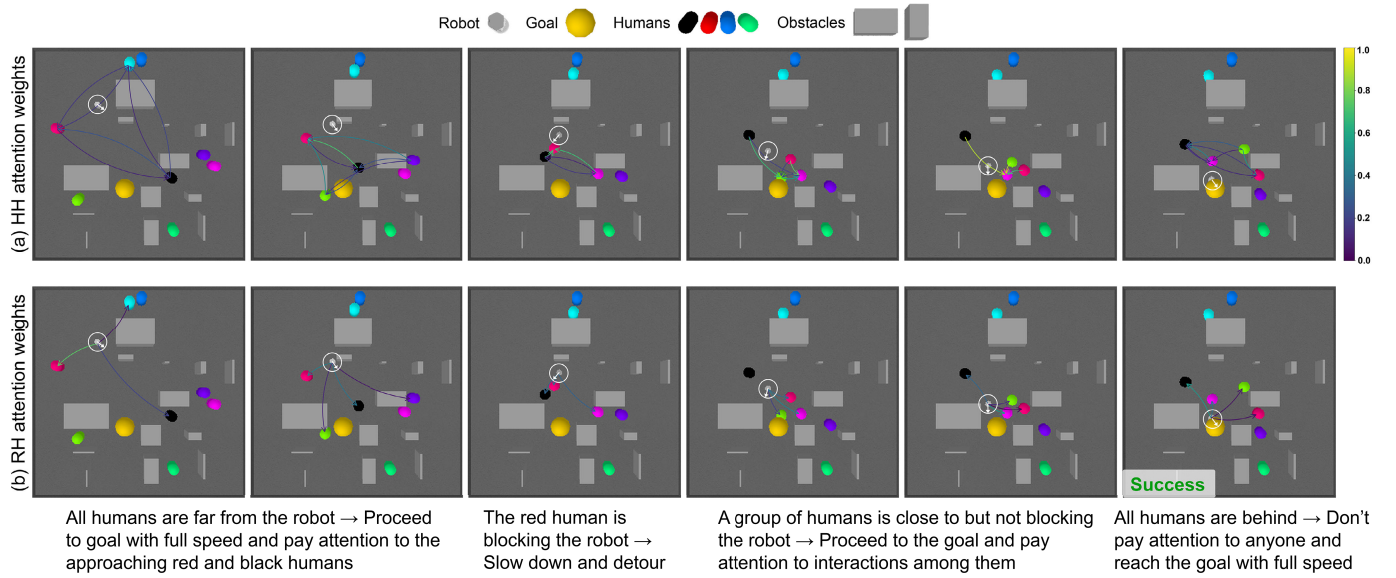


Fig. 10. **Visualizations of HH and RH attention weights of HEIGHT.** The color of lines indicate the weight values (see the colorbar on the right). For simplicity, we only include the line if the weight value is greater than 0.1. We observe that the interactions that likely to affect robot navigation in the near future have high attention values. Examples include humans that are approaching the robot and humans that block the robot's path to the goal.

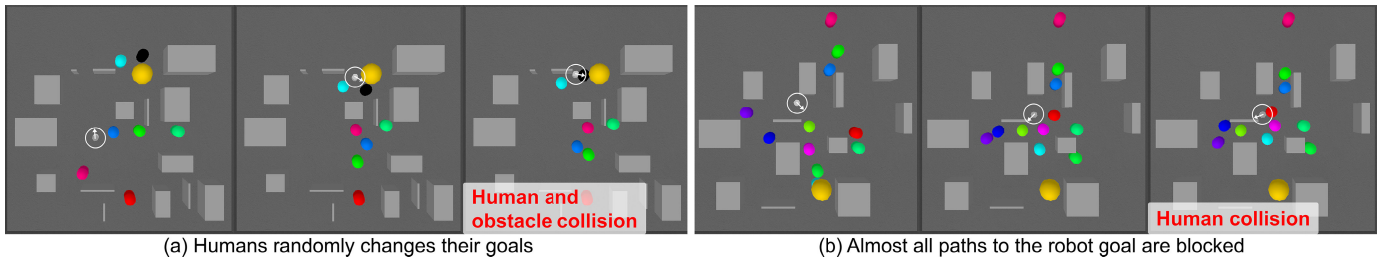


Fig. 11. **Failure cases of our method.** (a) The black human was going downwards for a while, but suddenly changes its direction toward the robot. (b) All efficient paths toward the goal are blocked or will soon be blocked by human crowds.

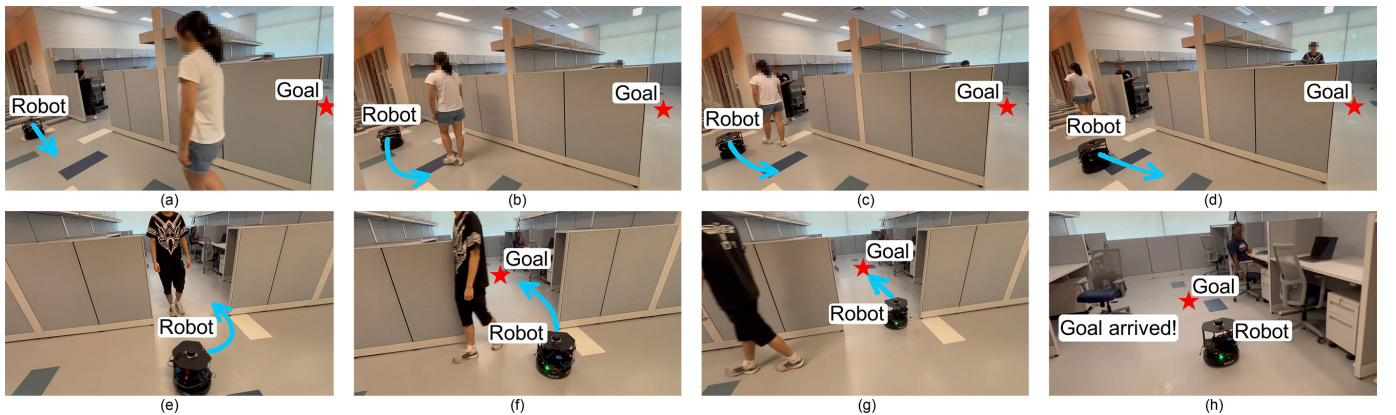


Fig. 12. **A testing episode of our method in the real Hallway environment.** The blue arrow denotes the robot path that results from its actions. The red star denotes the goal position. The turtlebot avoids two pedestrians, one after another in a narrow corridor, enters a narrow doorway, and arrives at the goal.

underfitting problem, where the robot collides with obstacles or goes back and forth without arriving at the goal. On the contrary, by using different representations of humans and obstacles, our method is able to explore the different strategies to avoid them through trial and error during RL training. For example, in Fig. 12(e)-(g), when turning into the narrow doorway, the robot takes a wide turn to yield enough space

so that the human can exit the doorway. However, the robot did not exhibit such prediction-aware behaviors when avoiding obstacles. Consequently, the robot demonstrates high success rates in Table IV. Qualitatively, the robot is able to safely pass a human in a narrow corridor (Fig. 12 (a)-(d) in Appendix), takes a wide turn to give a human enough room, and enters an extremely narrow doorway (Fig. 12 (e)-(h)).

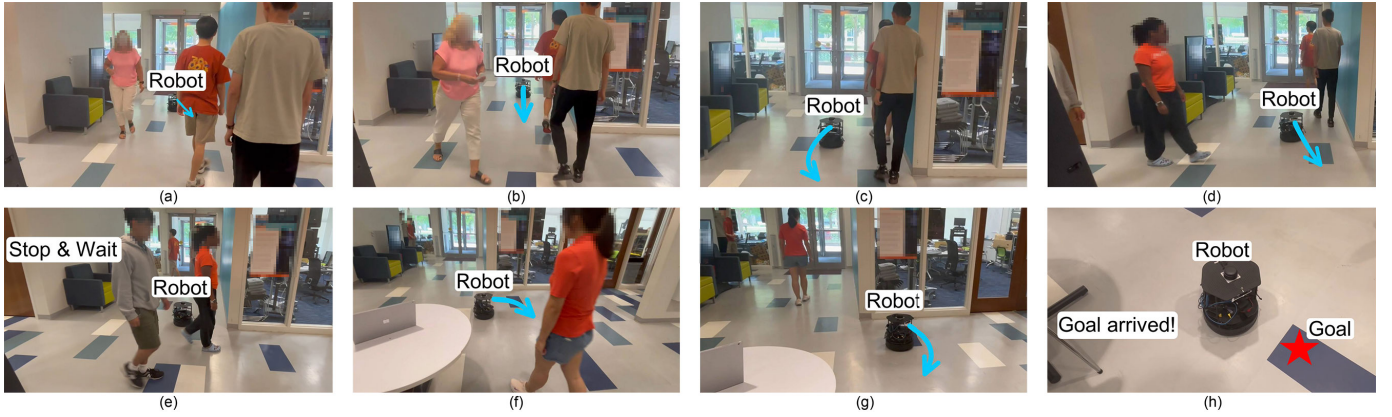


Fig. 13. **A testing episode of our method in the real Lounge environment.** The turtlebot avoids multiple groups of people who pass each other in different heading directions, avoids the walls and furnitures, and arrives at the goal.

In the less constrained yet more crowded Lounge environment, DRL-VO has the highest collision rate due to its underfitting problem. In addition, since only DRL-VO requires human velocities as inputs, the noises of human velocity estimation exaggerates its problem. ROS navigation stack and ours have the same success and collision rate, because the Lounge environment has enough space for the navigation stack to apply a “stop and wait” strategy for humans and obstacles. However, as a cost, “stop and wait” takes a longer average time to arrive at goals as shown in the last column of Table IV. Different from this naïve strategy, HEIGHT adapts its navigation behaviors based on different types of spatio-temporal interactions. For example, in Fig. 13 (a)-(c) in Appendix, since pedestrians walking in opposite directions are crossing each other, the robot first turns left to avoid the lady on its right, and then turns right to avoid the two males on its left. Then, in Fig. 13 (d)-(e) and (f)-(g), the robot chooses actions that deviate from the current and intended paths of pedestrians, walls, and the table to safely arrive at the goal. The failure cases of both methods are caused by changes in the arrangement of tables and chairs, while the fake point clouds are generated from a map with outdated furniture positions. This issue can be mitigated by replacing fake point clouds with real-time point clouds from sensors.

C. Deployment Results

The behavior mismatch between ORCA humans in simulation and real pedestrians is exaggerated during deployment. Since real pedestrians are usually risk-averse and conservative when interacting with the robot, all simulated humans react to the robot with a larger safety space. If the simulated humans out of robot’s view are about to collide with the robot, they freeze until they are far enough from the robot. To conquer perception and dynamics gaps, we strictly duplicate ZED 2i camera’s sensor range (120° field-of-view (FoV) and humans are only detectable 1 ~ 15 meters from the camera) and use a kinematic bicycle model with measured delays in simulation.

As shown in Fig. 7 and Fig. 8, during deployment, our method has learned an adaptive policy that accommodates the robot’s limitations. When the crowd is sparse, the robot circles

around pedestrians with a large safety distance to keep them in its FoV as long as possible. When the crowd is dense, the robot stops temporarily and keeps adjusting its orientation to find a direction with enough space to move forward, and resumes right after the pedestrians pass the robot. In the outdoor environment where pedestrians form a two-way traffic, the robot shows a smart strategy by merging into the traffic that moves in the same direction. These observations demonstrate that our method has learned an adaptive policy that is robust to sim2real gaps and can navigate safely and efficiently with unpredictable humans.

VII. DISCUSSIONS, LIMITATIONS, AND FUTURE WORK

In this section, we reflect on the key components of our framework, discuss the limitations of our approach, and propose directions for future research.

A. Sim2Real Through Real2Sim

Lessons learned: To overcome sim2real gaps, the design of simulation pipelines needs to be guided by the constraints of hardware and environments in the real-world. On one hand, we determine the input representation of HEIGHT based on what could be obtained from sensors and perception modules and how accurate they are. We find that intermediate features, such as detected human positions and processed point clouds, reduce sim2real gaps. On the other hand, we also ensure the consistency of the simulation and real-world, such as the robot dynamics model, whenever possible. These design choices allow our policy to generalize to different simulation environments and be deployed in challenging real-world scenarios.

Limitation and future work: Although we have minimized the sim2real gaps through real2sim, certain gaps still exist. Before sim2real transfer, the policy needs to be finetuned in simulations that mimic the target environments (Fig. 9). To further align the simulation and the real-world, we plan to explore the following directions for future work: (1) developing a more natural pedestrian model to replace the ORCA humans in the simulator, (2) revising our pipeline to enable failure detections and self-supervised RL fine-tuning in the real-world [62], [63], (3) using a small amount of real-world

data to automatically optimize the parameters of our simulator to match the real-world environment [64], [65].

B. Scene Representation

Lessons learned: A good scene representation should be tailored to the needs of its downstream task. Besides the above sim2real considerations, our scene representation is split due to the different nature of humans and obstacles for robot collision avoidance. The size of humans are small and their shapes are simple. Therefore, to avoid humans, the robot only needs to treat them as moving circles. In contrast, obstacles have larger and more complicated shapes. The part of the obstacles that faces the robot is more important for collision avoidance. Therefore, point clouds are the most intuitive way to represent obstacles. Our experiments in Sec. V show that this split scene representation reduces robot collision avoidance with both dynamic and static obstacles.

Limitation and future work: A side effect of our scene abstraction is the loss of detailed information such as gaits of humans and 3D shapes of obstacles. Another side effect is the cascading errors between the perception modules and robot policy, such as inaccurate human detections or robot localization. A future direction is the joint optimization of the whole robotic stack from perception to control [66], [67].

C. Structured Neural Network

Lessons learned: Model-based approaches require low-fidelity data yet heavily rely on assumptions. In contrast, end-to-end learning approaches need few assumptions yet require high-fidelity data. Our structured learning method combines the best of both worlds: It requires low-fidelity data yet relies on minimal assumptions. By injecting structures into the neural network, we decompose a complex problem into smaller and relatively independent components. Note that our decomposition does not break the gradient flow, which keeps HEIGHT end-to-end trainable. We propose a principled way for network architecture design, increasing the transparency of these end-to-end neural networks. Our experiments demonstrate that the structured network outperforms both model-based methods and RL-based methods without structures, which empirically proves the effectiveness of structure learning for interactive tasks with multiple heterogeneous participants.

Limitation and future work: The structured neural network utilizes the inductive bias behind human behaviors to solve specific problems. However, without re-training, the network cannot generalize to other tasks such as autonomous driving. In other fields such as computer vision and natural language processing, foundation models that are trained on large-scale diverse data have shown promise in generalization. Inspired by the standardized benchmarks of autonomous driving and robot manipulation [68], [69], we aim to contribute to the benchmarking efforts of human-centered tasks.

D. Training Method

Lessons learned: Deep RL is a promising tool to solve robotic problems that are beyond the capabilities of traditional

rule-based methods without large-scale real-world datasets. It enables the robot to explore the environment and learn meaningful behaviors through trial and error. Without heuristics or demonstrations, the robot has to collect reward signals to improve its policy. Consequently, simulation development with a randomization scheme and a good reward design are indispensable to the performance of our method.

Limitation and future work: Our method is subjective to the inherent limitations of RL, such as the need for simulators, low training data efficiency, and difficulties in long-horizon tasks. In addition, preventing performance degradation of the policy when inevitable distribution shifts happen, especially in real-world, is challenging. In future work, combining the RL policy with other planners, imitation learning, or foundation models may alleviate these problems [70].

E. Miscellaneous

Multi-robot navigation: This work focuses on single-robot navigation. Modeling robot-robot interactions in multi-robot navigation in crowded and constrained environments is an interesting future direction. To enable this, the extra robots and the interactions among all robots can be treated as extra nodes and spatial edges in the st-graph. Following our method, from the st-graph, robot-robot interactions can be modeled with self-attention or other methods, which serves as another submodule in each robot's policy network.

Social awareness: The attention-based interaction framework presents a promising foundation for some social behaviors (e.x. treating humans and obstacles differently). However, HEIGHT focuses on optimizing task success and efficiency, and thus lacks explicit mechanisms and evaluation metrics for social awareness [71]. Future work could integrate social norm as constraints or auxiliary objectives in RL training, as well as learning from human demonstrations or preference-based feedback. User studies can evaluate social awareness.

VIII. CONCLUSION

In this article, we proposed HEIGHT, a novel structured graph network architecture for autonomous robot navigation in dynamic and constrained environments. Our approach takes advantage of the graphical nature and decomposability of the constrained crowd navigation problem, introducing the following two key novelties. First, we split the human and static obstacle representations, which allows the robot to effectively reason about different collision avoidance strategies to deal with both. Second, we propose a heterogeneous st-graph to capture various types of interactions among the robot, humans, and obstacles. The heterogeneous st-graph guides the design of the HEIGHT network. Attention mechanism enables the robot to reason about the relative importance of each pairwise interaction, leading to adaptive and agile robot decisions during navigation. Our simulation benchmark shows that HEIGHT outperforms model-based and learning-based methods in terms of collision avoidance and navigation time. HEIGHT also demonstrates improved generalization when human and obstacle densities vary. HEIGHT is

seamlessly transferred from simulation to real-world navigation scenarios, showcasing its robustness and deployability.

Our work suggests that reasoning about subtle spatio-temporal interactions is an essential step toward smooth human-robot interaction. Furthermore, our work highlights the significance of uncovering the inherent structure of complex problems and injecting these structures into learning frameworks to solve the problems in a principled manner.

APPENDIX

A. Simulation Experiments–Baselines

- **Dynamic window approach (DWA)** [4] searches for the optimal velocity that brings the robot to the goal with the maximum clearance from any obstacle. DWA is a model-based method that only considers the current state. DWA represents both humans and static obstacles as groups of small circles. We re-mapped the output of DWA to match the turtlebot kinematics and our action space.
- **Optimal Reciprocal Collision Avoidance (ORCA)** [13] optimizes for the optimal velocity that brings the robot to the goal by assuming that agents avoid each other under the reciprocal rule. ORCA is a model-based method that only considers the current state. In addition, ORCA represents both humans as circles and static obstacles as line segments. We re-mapped the output of ORCA to match the unicycle kinematics of our robot.
- **A*+CNN** [6] is a hybrid method with sample-based planning and RL. With a map of the environment, A* is the global planner and generates 6 waypoints in the beginning of an episode. The inputs to the RL policy are a 2D LiDAR point cloud, the robot state, the waypoints, and the robot goal. No human detections are used and human features are included in the point cloud. The point cloud is passed through a CNN, whose output is concatenated with the embedding of other inputs. In addition to the reward function in Eq. 1, the robot receives a heading direction award if it moves towards the goal while being collision-free.
- **Decentralized structural RNN (DS-RNN)** [8] is an RL-based method that represents static obstacles as groups of small circles. In network architecture, DS-RNN only contains the RH attention and the GRU.
- **DRL-VO** [28] is a hybrid method with a pure pursuit algorithm as the global planner and an RL policy as the local planner. The inputs to the planners are the positions and velocities of humans encoded in an occupancy map, a 2D LiDAR point cloud, and the robot state. The RL network is a CNN without explicit interaction modeling.
- **Homogeneous graph attention network (HomoGAT)** [45] is RL-based and splits human and obstacle inputs. However, HomoGAT does not differentiate between 3 types of nodes and 3 types of edges in policy network. Instead, HomoGAT uses a single self-attention network to weigh humans, the robot, and the obstacle point cloud and feed the weighted sum of all embeddings into a GRU. HomoGAT is similar to Chen et al. [7] and Liu et al. [51] but the input, output, and training algorithm are kept the same as our method for a fair comparison.

TABLE V

COMPARISON OF TRAINING AND INFERENCE TIME IN THE TRAINING ENVIRONMENT. ALL RL METHODS ARE TRAINED FOR 2×10^8 STEPS. THE INFERENCE TIME IS AVERAGED OVER 500 TESTING STEPS

Method	Training (hours)	Inference (ms/step)
DWA [4]	N/A	1.5
ORCA [13]	N/A	0.3
A*+CNN [6]	47.2	2.0
DS-RNN [8]	49.0	1.0
DRL-VO [28]	111.9	2.3
HomoGAT [45]	47.7	2.2
HEIGHT (ours)	49.7	2.4

B. Simulation Experiment Setup

1) *Training Details:* We train all RL methods, including all baselines and ablations except DWA, for 2×10^8 timesteps with a learning rate 5×10^{-5} . The learning rate decays at a linear rate with respect to training timesteps. To accelerate and stabilize training, we run 28 parallel environments to collect the robot's experiences. At each policy update, 30 steps of 6 episodes are used.

2) *Resource Usage:* In simulation experiments, we train and test all methods in a commercial desktop computer with a Ubuntu 20.04 OS, an Intel Core i9-13900K processor, 32 GB RAM, and a NVIDIA RTX 4090 GPU. We report the training and testing time in simulation with the above computer in Table V. For training time, all RL methods except DRL-VO are similarly fast, while DRL-VO is the slowest due to its high-dimensional occupancy map input and the larger 2D CNN network. For inference time, HEIGHT is the slowest because the two attention networks for spatial reasoning and the GRU for temporal reasoning increases computation cost. Thus, we observe a tradeoff between performance and time efficiency. In future work, the inference latency can be reduced by (1) combining the state of multiple humans in the same group and (2) deploying transformer acceleration techniques such as key-value caching or sparse attention.

C. Simulation Experiments–Additional Insights

Effectiveness of RL planning: In Table II, DWA and ORCA are model-based approaches that determines robot actions based on only the current state, which leads to the worst performances. Thus, reactive policies are not sufficient to solve our problem, justifying the necessity of long-sighted planning. As a step forward, A*+CNN combines planning and RL, yet the planner relies on occupancy maps without humans, reducing the optimality of planned waypoints and negatively affecting overall navigation. On the other hand, RL learns to maximize the expected long-term returns based on both the current and historical states of all components in the scene. Consequently, the remaining 3 RL methods, especially HomoGAT and HEIGHT, outperform A*+CNN in most metrics and in most environments. Thus, to ensure the best navigation performance, it is important to optimize the entire planning system based on the task instead of optimizing only a part of it.

Difficulty of scenarios: From Table II and Table III, we observe that, besides the overfitted DS-RNN, all methods show

the same trend of performance change in the 5 environments: *Less crowded* > *Less constrained* > *More constrained* > *More crowded*. Obviously, adding more humans or obstacles increases task difficulty. But interestingly, the change in the number of humans has a larger effect on the task difficulty than the change in the number of obstacles. This phenomenon shows that avoiding collisions with humans is more difficult than avoiding obstacles in nature.

D. Real-World Experiment Setup

We train the sim2real policies in the simulators of a hallway (Fig. 9(b) in Appendix), a lounge (Fig. 9(c)), and an atrium (Fig. 9(a)) for 2×10^8 timesteps with a decaying learning rate 5×10^{-5} . The distance between the starting and the goal position of the robot ranges from 6m to 11m. The pedestrians were told to react naturally to the robot based on their own preferences. In some testing episodes, other pedestrians who were unaware of our experiment also engaged with the robot.

We use the following two sets of robot hardware:

- In Hallway and Lounge environments, we use a TurtleBot 2i equipped with an RPLIDAR-A3 laser scanner, an Intel RealSense tracking camera T265, a Nvidia Jetson Xavier board, and a remote host computer. We first use the ROS gmapping package to create a map of the environment. Then, we process the map to combine small obstacles and eliminate noises. To detect human positions from LiDAR point clouds, we use an off-the-shelf people detector, DR-SPAAM [53]. Since DR-SPAAM tends to treat obstacles as false positive humans, we mitigate this issue by removing the point clouds that belong to pre-mapped static obstacles from the LiDAR readings, then feed the remaining point clouds that only contain humans to DR-SPAAM. We use the T265 camera to obtain the pose of the robot (p_x, p_y, θ) and the robot wheel encoder to obtain its velocity (u_x, u_y). The host computer runs the robot policies and perception modules, while the TurtleBot only receives and sends ROS messages. The host computer and TurtleBot communicate through ROS.
- In Atrium and Outdoor environment, we use a Clearpath Jackal robot equipped with an ZED 2i camera and an Ouster LiDAR. The system is built using ROS. The software stack is similar to the TurtleBot except the following differences: We use ZED 2i to detect the positions of humans and the pose of the robot. The pre-built map and robot localization are obtained from ENML algorithm [72]. All computations are executed entirely onboard, utilizing an Intel i7-9700TE CPU and an NVIDIA RTX A2000 GPU.

ACKNOWLEDGMENT

The authors thank Zhe Huang for helpful discussions and the code of the A* planner. They thank Chen Tang, Aamir Hasan, and Rutav Shah for feedback on article drafts. They also thank colleagues at the University of Illinois Urbana-Champaign and at The University of Texas at Austin who participated in real-world experiments. Any opinions, findings, and conclusions expressed in this material are those

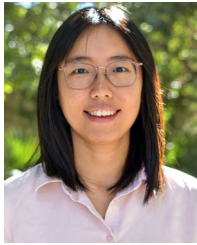
of them and do not necessarily reflect the views of the sponsors.

REFERENCES

- [1] C. Chen, Y. Liu, S. Kreiss, and A. Alahi, "Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2019, pp. 6015–6022.
- [2] S. Liu et al., "Intention aware robot crowd navigation with attention-based interaction graph," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 12015–12021.
- [3] W. Wang, R. Wang, L. Mao, and B.-C. Min, "NaviSTAR: Socially aware robot navigation with hybrid spatio-temporal graph transformer and preference learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2023, pp. 11348–11355.
- [4] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robot. Autom. Mag.*, vol. 4, no. 1, pp. 23–33, Mar. 1997.
- [5] J. van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2008, pp. 1928–1935.
- [6] C. Pérez-D'Arpino, C. Liu, P. Goebel, R. Martín-Martín, and S. Savarese, "Robot navigation in constrained pedestrian environments using reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 1140–1146.
- [7] C. Chen, S. Hu, P. Nikdel, G. Mori, and M. Savva, "Relational graph learning for crowd navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 10007–10013.
- [8] S. Liu, P. Chang, W. Liang, N. Chakraborty, and K. Driggs-Campbell, "Decentralized structural-RNN for robot crowd navigation with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 3517–3524.
- [9] B. Chen et al., "Socially aware object goal navigation with heterogeneous scene representation learning," *IEEE Robot. Autom. Lett.*, vol. 9, no. 8, pp. 6792–6799, Aug. 2024.
- [10] M. Hoy, A. S. Matveev, and A. V. Savkin, "Algorithms for collision-free navigation of mobile robots in complex cluttered environments: A survey," *Robotica*, vol. 33, no. 3, pp. 463–497, Mar. 2015.
- [11] A. V. Savkin and C. Wang, "Seeking a path through the crowd: Robot navigation in unknown dynamic environments with moving obstacles based on an integrated environment representation," *Robot. Auto. Syst.*, vol. 62, no. 10, pp. 1568–1580, Oct. 2014.
- [12] C. Mavrogiannis et al., "Core challenges of social robot navigation: A survey," 2021, *arXiv:2103.05668*.
- [13] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*, vol. 70. Berlin, Germany: Springer, 2011, pp. 3–19.
- [14] D. Helbing and P. Molnár, "Social force model for pedestrian dynamics," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 51, no. 5, pp. 4282–4286, May 1995.
- [15] L. Huber, J.-J. Slotine, and A. Billard, "Avoiding dense and dynamic obstacles in enclosed spaces: Application to moving in crowds," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3113–3132, Oct. 2022.
- [16] C. Mavrogiannis, K. Balasubramanian, S. Poddar, A. Gandra, and S. S. Srinivasa, "Winding through: Crowd navigation via topological invariance," *IEEE Robot. Autom. Lett.*, vol. 8, no. 1, pp. 121–128, Jan. 2023.
- [17] Open Source Robotics Foundation. (2007). *ROS Navigation Stack*. Accessed: Nov. 11, 2024. [Online]. Available: <http://wiki.ros.org/navigation>
- [18] M. Dobrevski and D. Skočaj, "Dynamic adaptive dynamic window approach," *IEEE Trans. Robot.*, vol. 40, pp. 3068–3081, 2024.
- [19] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 6252–6259.
- [20] P. Trautman and A. Krause, "Unfreezing the robot: Navigation in dense, interacting crowds," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2010, pp. 797–803.
- [21] Z. Liu, W. Na, C. Yao, C. Liu, and Q. Chen, "Relaxing the limitations of the optimal reciprocal collision avoidance algorithm for mobile robots in crowds," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5520–5527, Jun. 2024.
- [22] P. Long, W. Liu, and J. Pan, "Deep-learned collision avoidance policy for distributed multiagent navigation," *IEEE Robot. Autom. Lett.*, vol. 2, no. 2, pp. 656–663, Apr. 2017.

- [23] L. Tai, J. Zhang, M. Liu, and W. Burgard, "Socially compliant navigation through raw depth inputs with generative adversarial imitation learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Jun. 2018, pp. 1111–1117.
- [24] Z. Xie, P. Xin, and P. Dames, "Towards safe navigation through crowded dynamic environments," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 4934–4940.
- [25] A. Pokle et al., "Deep local trajectory replanning and control for robot navigation," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2019, pp. 5815–5822.
- [26] H. Karnan et al., "Socially CompliAnt navigation dataset (SCAND): A large-scale dataset of demonstrations for social navigation," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11807–11814, Oct. 2022.
- [27] R. Chandra, H. Karnan, N. Mehr, P. Stone, and J. Biswas, "Multi-agent inverse reinforcement learning in real world unstructured pedestrian crowds," 2024, *arXiv:2405.16439*.
- [28] Z. Xie and P. Dames, "DRL-VO: Learning to navigate through crowded dynamic scenes using velocity obstacles," *IEEE Trans. Robot.*, vol. 39, no. 4, pp. 2700–2719, Aug. 2023.
- [29] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2017, pp. 285–292.
- [30] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 3052–3059.
- [31] Y. Chen, C. Liu, B. E. Shi, and M. Liu, "Robot navigation in crowds by graph convolutional networks with attention learned from human gaze," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2754–2761, Apr. 2020.
- [32] Y. Yang, J. Jiang, J. Zhang, J. Huang, and M. Gao, "ST2: Spatial-temporal state transformer for crowd-aware autonomous navigation," *IEEE Robot. Autom. Lett.*, vol. 8, no. 2, pp. 912–919, Feb. 2023.
- [33] Y.-J. Mun, M. Itkina, S. Liu, and K. Driggs-Campbell, "Occlusion-aware crowd navigation using people as sensors," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 12031–12037.
- [34] A. J. Sathiamoorthy, J. Liang, U. Patel, T. Guan, R. Chandra, and D. Manocha, "DenseCAvoid: Real-time navigation in dense crowds using anticipatory behaviors," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 11345–11352.
- [35] L. Chen et al., "Decentralized multi-robot navigation coupled with spatial-temporal RetNet based on deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 10289–10296.
- [36] D. Dugas, O. Andersson, R. Siegwart, and J. J. Chung, "NavDreams: Towards camera-only RL navigation among humans," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 2504–2511.
- [37] Z. Zheng, C. Cao, and J. Pan, "A hierarchical approach for mobile robot exploration in pedestrian crowd," *IEEE Robot. Autom. Lett.*, vol. 7, no. 1, pp. 175–182, Jan. 2022.
- [38] N. Tsou et al., "SEAN 2.0: Formalizing and generating social situations for robot navigation," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11047–11054, Oct. 2022.
- [39] X. Puig et al., "Habitat 3.0: A co-habitat for humans, avatars and robots," in *Proc. Int. Conf. Learn. Represent.*, Mali, 2023.
- [40] L. Kästner et al., "Demonstrating arena 3.0: Advancing social navigation in collaborative and highly dynamic environments," in *Robotics: Science and Systems*, Jul. 2024.
- [41] R. Martín-Martín et al., "JRDB: A dataset and benchmark of egocentric robot visual perception of humans in built environments," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 6748–6765, Jun. 2023.
- [42] N. Hirose, D. Shah, A. Sridhar, and S. Levine, "SACSoN: Scalable autonomous control for social navigation," *IEEE Robot. Autom. Lett.*, vol. 9, no. 1, pp. 49–56, Jan. 2024.
- [43] H. Yang, C. Yao, C. Liu, and Q. Chen, "RMRL: Robot navigation in crowd environments with risk map-based deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 8, no. 12, pp. 7930–7937, Dec. 2023.
- [44] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2025, pp. 5998–6008.
- [45] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent.*, 2018.
- [46] A. Vemula, K. Muelling, and J. Oh, "Social attention: Modeling attention in human crowds," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 1–7.
- [47] Y. Huang, H. Bi, Z. Li, T. Mao, and Z. Wang, "STGAT: Modeling spatial-temporal interactions for human trajectory prediction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6271–6280.
- [48] A. Hasan, P. Sriram, and K. Driggs-Campbell, "Meta-path analysis on spatio-temporal graphs for pedestrian trajectory prediction," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 617–624.
- [49] E. Leurent and J. Mercat, "Social attention for autonomous decision-making in dense traffic," in *Proc. Mach. Learn. Auto. Driving Workshop NeurIPS*, 2019.
- [50] S. Liu, P. Chang, H. Chen, N. Chakraborty, and K. Driggs-Campbell, "Learning to navigate intersections with unsupervised driver trait inference," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 3576–3582.
- [51] L. Huajian, D. Wei, M. Shouren, W. Chao, and G. Yongzhuo, "Sample-efficient learning-based dynamic environment navigation with transferring experience from optimization-based planner," *IEEE Robot. Autom. Lett.*, vol. 9, no. 8, pp. 7055–7062, Aug. 2024.
- [52] Z. Liu, Y. Zhai, Z. Liu, G. Wang, Y. Miao, and H. Wang, "Graph relational reinforcement learning for mobile robot navigation in large-scale crowded environments," in *Proc. IEEE Int. Conf. Intell. Transp. Syst. (ITSC)*, Jun. 2023, vol. 24, no. 8, pp. 8776–8787.
- [53] D. Jia, A. Hermans, and B. Leibe, "DR-SPAAM: A spatial-attention and auto-regressive model for person detection in 2D range data," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 10270–10277.
- [54] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
- [55] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2017, pp. 3645–3649.
- [56] C. Mavrogiannis et al., "Core challenges of social robot navigation: A survey," *ACM Trans. Hum.-Robot Interact.*, vol. 12, no. 3, pp. 1–39, 2023.
- [57] A. Jain, A. R. Zamir, S. Savarese, and A. Saxena, "Structural-RNN: Deep learning on spatio-temporal graphs," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 5308–5317.
- [58] X. Ma, J. Li, M. J. Kochenderfer, D. Isele, and K. Fujimura, "Reinforcement learning for autonomous driving with latent state inference and spatial-temporal relationships," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 6064–6071.
- [59] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [60] E. Coumans and Y. Bai, *Pybullet, a Python Module for Physics Simulation for Games, Robotics and Machine Learning*. Accessed: Nov. 1, 2024. [Online]. Available: <http://pybullet.org>
- [61] Z. Hu et al., "Composablenav: Instruction-following navigation in dynamic environments via composable diffusion," in *Proc. Conf. Robot Learn.*, 2025.
- [62] H. Zhu et al., "The ingredients of real world robotic reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, 2020.
- [63] P. Chang, S. Liu, T. Ji, N. Chakraborty, D. L. McPherson, and K. Driggs-Campbell, "A data-efficient visual-audio representation with intuitive fine-tuning for voice-controlled robots," in *Proc. Conf. Robot Learn.*, 2023.
- [64] Y. Du, O. Watkins, T. Darrell, P. Abbeel, and D. Pathak, "Auto-tuned sim-to-real transfer," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, p. 1290.
- [65] V. Lim et al., "Real2Sim2Real: Self-supervised learning of physical single-step dynamic actions for planar robot casting," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 8282–8289.
- [66] Y. Hu et al., "Planning-oriented autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 17853–17862.
- [67] H. Wang et al., "Mosaic: A modular system for assistive and interactive cooking," in *Proc. 2nd Workshop Mobile Manipulation Embodied Intell.*, 2024, pp. 1–5.
- [68] H. Caesar et al., "NuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles," 2021, *arXiv:2106.11810*.
- [69] O. X-Embodiment Collaboration et al., "Open X-embodiment: Robotic learning datasets and RT-X models," 2023, *arXiv:2310.08864*.
- [70] A. H. Raj et al., "Rethinking social robot navigation: Leveraging the best of two worlds," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 16330–16337.
- [71] A. Francis et al., "Principles and guidelines for evaluating social robot navigation algorithms," *ACM Trans. Hum.-Robot Interact.*, vol. 14, no. 2, pp. 1–65, Jun. 2025.

- [72] J. Biswas and M. Veloso, "Episodic non-Markov localization: Reasoning about short-term and long-term features," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2014, pp. 3969–3974.



Shuijing Liu received the B.S. and Ph.D. degrees in electrical and computer engineering from the University of Illinois Urbana–Champaign, in 2018 and 2024, respectively. She is currently a Post-Doctoral Researcher with the Computer Science Department, The University of Texas at Austin. She was the Advisory Board of the Artificial Intelligence, startup named Unicorn.ai. Her research interests include intersection of human-robot interaction and robot learning, with the goal of building a trustworthy partnership between humans, and robots in the wild.



Haochen Xia is currently pursuing the B.S. degree in computer engineering, with a minor in mathematics, with the University of Illinois Urbana–Champaign, Urbana, IL, USA. He is an Undergraduate Research Assistant with the Human-Centered Autonomy Laboratory and the ECE Department, working under Prof. Katie Driggs-Campbell and Prof. David M. Nicol. His research interests include human–robot interaction, cognitive AI, multimodal perception, and learning-based motion planning.



Fatemeh Cheraghi Pouria received the B.S. degree in mechanical engineering from the Sharif University of Technology, Tehran, Iran, in 2022. She is currently pursuing the Ph.D. degree in electrical and computer engineering with the University of Illinois Urbana–Champaign. Her current research interests include human trajectory prediction and robot navigation.



Kaiwen Hong received the B.E. degree in electrical engineering from Zhejiang University, Hangzhou, China, and the B.S. degree in electrical engineering from the University of Illinois Urbana–Champaign, Urbana, IL, USA, in 2021, where he is currently pursuing the Ph.D. degree in electrical and computer engineering. His current research interests include robot learning and human-centered robotics.



Neeloy Chakraborty (Graduate Student Member, IEEE) received the B.S. and M.S. degrees in computer engineering from the University of Illinois Urbana–Champaign in 2013 and 2021, respectively, where he is currently pursuing the Ph.D. degree in electrical and computer engineering. His current research interests include foundation model evaluation and field robotics.



Zichao Hu (Member, IEEE) received the B.S. degree in computer engineering from the University of Virginia, Virginia, USA, in 2022. He is currently pursuing the Ph.D. degree in computer science with The University of Texas at Austin, Texas, USA. His current research interests include diffusion policy for social navigation and whole-body control for humanoid robot object interaction.



Joydeep Biswas (Member, IEEE) received the B.Tech. degree in engineering physics from Indian Institute of Technology, Bombay, in 2008, and the M.S. and Ph.D. degrees in robotics from Carnegie Mellon University, in 2010 and 2014, respectively. He is currently an Associate Professor with the Computer Science Department, The University of Texas at Austin and a Visiting Professor with Nvidia. His research interests include robot perception, motion planning, control systems, AI, and deployed robot systems.



Katherine Driggs-Campbell (Member, IEEE) received the B.S.E. degree in electrical engineering from Arizona State University, in 2012, and the M.S. and Ph.D. degrees in electrical engineering and computer sciences from the University of California, Berkeley, in 2015 and 2017, respectively. She is currently an Associate Professor with the Electrical and Computer Engineering Department, University of Illinois Urbana–Champaign. Her research interests are developing tools for autonomous robotics that operate out in the real world with people, which is

drawn from the fields of human-robot interaction, optimization, learning and AI, and control theory.